



htl donaustadt

HTL-Donaustadt
Höhere Abteilung für Informatik



DIPLOMARBEIT



Plant Up!

Smart Plant Monitoring with IoT and Gamification

Ausgeführt im Schuljahr 2025/26 von:

Arun Sherzai
Abd-Ulrahman Behari
Richard Onuoha
Dennis Jin

Betreuer/Betreuerin:

Mag. Dr. Putzinger

Wien, 1. März 2026

«««< Updated upstream **STATEMENT OF AUTHORSHIP**

I hereby certify that this diploma thesis has been composed by myself, and describes my own work, unless otherwise acknowledged in the text. All references and verbatim extracts have been quoted, and all sources of information have been specifically acknowledged.

Wien, 1. März 2026

Arun SHERZAI

Signature

Abd-Ulrahman BEHARI

Signature

Richard ONUOHA

Signature

Dennis JIN

Signature

===== Author's Declaration

I hereby declare that I have written this thesis independently and used only the tools and resources listed. Any sections taken verbatim or paraphrased from other works (including databases) are clearly cited following academic referencing rules. This declaration also applies to images, tables, sketches, and drawings used in the thesis.

For assistance, I utilized generative AI tools such as ChatGPT, Grammarly Go, and this document. The use of these tools has been fully disclosed, including versions used. This document has been generated for academic purposes only and should not be considered as legal or expert advice. While every effort has been made to ensure the accuracy and reliability of the information presented, the author and publisher assume no responsibility for any errors, omissions, or any consequences resulting from the use of this document.

Portions of this text have been structurally enhanced with the assistance of ChatGPT. The tool was used solely for formatting, organization, and language clarity, and the core content and its integrity remains with the author.

Any opinions, findings, or conclusions expressed herein are those of the author and do not necessarily reflect the views of any organizations or entities.

The inclusion of external sources, links, or references does not imply endorsement, and the responsibility for verifying their contents rests solely with the reader. For specific guidance tailored to your circumstances, please consult a qualified professional.

Arun Sherzai

Abd-Ulrahman Behari

Richard Onuoha

Dennis Jin

Wien, 1. März 2026»»»> Stashed changes

Danksagungen

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquid ex ea commodi consequat.

Abstract

Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Subsubsection 1

Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Subsubsection 2

Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Inhaltsverzeichnis

Chapter 1

Introduction

The "Plant Boom" vs. The Reality

The indoor plant market experienced a massive rise during the early 2020s, with people spending more money on indoor plants. From \$1.31 billion in 2019 to \$2.17 billion by 2021 [?]. This "boom" saw household participation in indoor gardening reach a four-year high of 38.1 million households [?]. However, even with this rise in popularity, the reality is that people are not very good at taking care of plants. While demand rose by 18%, failure rates remained constant: approximately 40% of plants perish within the supply chain before reaching a shelf [?], and another 35% die shortly after entering a customer's home [?]. This indicates that the industry's economic "boom" is significantly fueled by a replacement cycle where consumers repeatedly purchase new plants to compensate for those lost.

What is "Plant Up!" ?

Plant Up! is a smart gardening system that aims to solve this problem by providing users with the tools and knowledge they need to keep their plants alive. It combines IoT sensors with a mobile application to create a seamless and user-friendly experience [?]. The system monitors soil moisture, light, and temperature in real-time, and provides users with actionable insights and guidance on how to care for their plants.

What "Plant Up!" offers

Plant Up! [?] offers a range of features designed to help users keep their plants alive. These include:

- Real-time monitoring of soil moisture, light, and temperature

- Actionable insights and guidance on how to care for plants
- A comprehensive database of plants and their care requirements
- A community forum for users to share tips and advice

Chapter 2

Combining Nature and Technology: How Sensor-Based and Data-Driven Technologies Enhance Plant Monitoring and Care

Author: Arun Sherzai

2.1 Introduction

Houseplants are a common feature in modern households, with the global indoor plant market valued at approximately 21 billion USD in 2025 [?]. Yet maintaining them over longer periods remains challenging for many people. One of the most common reasons is improper care rather than neglect. In particular, horticultural experts identify overwatering as the leading cause of houseplant failure, frequently resulting in yellowing leaves, wilting, and root rot [?]. This indicates a fundamental issue: plants often suffer because their actual needs are misunderstood.

A key reason for this misunderstanding lies in the limitations of human judgment. Soil moisture is commonly assessed by touching the surface of the soil, even though moisture levels can differ significantly between the upper layer and the root zone [?]. As a result, soil may appear dry while deeper layers remain saturated, which increases the risk of root diseases and root rot [?]. These processes occur below the surface and are therefore difficult to detect without technical support.

Human perception of light conditions is similarly unreliable. The visual system adapts to changing brightness levels, which often leads to an overestimation of the usable light available in indoor environments [?]. A room may appear bright to humans while still providing insufficient light for healthy plant growth.

On the other side, sensor technology and Internet of Things (IoT) applications are becoming increasingly widespread. The global smart home market is experiencing strong

growth, driven mainly by automation in areas such as lighting, climate control, and energy management [?, ?]. Despite this development, plant care in private households remains largely intuition-based.

This gap motivates the development of Plant Up!.

The primary objective of this thesis is to show that effective plant monitoring does not require expensive or specialized equipment. Using affordable, consumer-grade hardware such as soil moisture sensors, digital environmental sensors, and a microcontroller-based data acquisition system, this work demonstrates how common care mistakes can be reduced. In doing so, it aims to improve plant survival and provide users with a clearer understanding of the environmental conditions that influence plant health.

Based on the challenges outlined above, this thesis addresses the following research question:

How can a low-cost, sensor-based system be designed to provide objective environmental data and actionable care recommendations?

This question focuses on practical application rather than theoretical considerations. The goal is not to replace horticultural expertise, but to support everyday plant owners by providing objective information that simplifies decision-making. By translating sensor measurements into clear feedback, the system helps users recognize when action is necessary and when intervention would be counterproductive.

The relevance of this research can be viewed from several perspectives. From an environmental perspective, this project helps save water. Many people water their plants based on routine or guessing, which often leads to waste. By using sensors, water is only used when the plant actually needs it, supporting a more sustainable approach [?, ?].

From a practical perspective, many plant owners are frustrated when their plants die despite regular care. By visualizing environmental conditions and offering clear care recommendations, successful plant care becomes easier, as users do not need detailed botanical knowledge.

The topic is also relevant from a technological perspective. Advances in microcontrollers, sensor accuracy, and wireless communication make it possible to implement functional monitoring systems using affordable consumer-level hardware. This thesis demonstrates how such technologies can be combined into an accessible Internet of Things (IoT) solution for everyday plant care.

To understand why these measurements are essential, the following chapter examines the environmental factors that directly influence plant growth and health.

2.2 Environmental Factors in Plant Growth

Plants are living systems, but they can be treated as measurable input-output systems in engineering practice. Light, water, temperature, and humidity are not “nice-to-have” background conditions. They are measurable variables with operating ranges and failure limits. When a variable stays outside its range, stress is not a surprise. It is the expected outcome. This chapter focuses on what can be measured and controlled, because Plant Up! must convert plant needs into sensor values and thresholds. [?]

2.2.1 Importance of Soil Moisture, Temperature, Light, and Humidity

Soil moisture

Soil is a porous material made of solids plus pore space filled with air and water. An ideal soil for plant growth is often described as 50% solids and 50% pore space, ideally split into equal parts air and water so roots get both oxygen and moisture. This is the physical reason why “slightly moist” often performs better than “always wet.” [?]

When pore spaces stay waterlogged, air is displaced and the oxygen supply to roots drops. Root function declines and nutrient uptake suffers, even while the pot appears wet. This oxygen deficit is the primary mechanism behind overwatering damage and explains why soggy soil is more dangerous than dry soil for most indoor plants. [?]

The opposite end of the spectrum is the *permanent wilting point*: the threshold where remaining soil moisture is physically unavailable to roots. Below this point the plant cannot recover through normal watering, because the accessible water reserve is fully depleted. For monitoring, volumetric water content (VWC) expresses soil water as a percentage of total soil volume, which provides a consistent, loggable metric. [?, ?]

Light

Light is the energy input that drives photosynthesis. Plants do not use all wavelengths equally, so the relevant band is Photosynthetically Active Radiation (PAR), typically defined as the 400–700 nm range. This distinction matters because “bright to humans” and “usable for plants” are not the same thing. [?]

The plant-focused intensity metric is PPFD (Photosynthetic Photon Flux Density), which counts photons in the PAR range arriving per area per second. Many low-cost systems instead measure illuminance in lux, which is weighted for human vision and therefore does not map cleanly to PPFD. Lux is still practical for relative comparisons within one setup, such as comparing a dark corner to a bright window. [?]

Temperature and humidity

Temperature controls how fast plant processes run. When it is too low, growth slows as biochemical reactions become sluggish. When it is too high, regulation mechanisms lose efficiency and heat damage risk increases. For indoor monitoring, temperature is a core variable because it directly affects water demand and interacts with humidity. [?]

Humidity links to transpiration, the process where water evaporates from leaf surfaces and creates an upward flow that transports dissolved minerals. When humidity is low, transpiration accelerates and the plant loses water faster, potentially outpacing root uptake even if the soil is moist. Because temperature changes the air's water-holding capacity, humidity must always be interpreted together with temperature to correctly assess the atmospheric demand on the plant. [?, ?]

Technical link: translating needs into metrics

The described processes depend on physical quantities, not on perception. Roots react to oxygen availability and water content, not to how soil feels. Leaves respond to photon availability and atmospheric drying power, not to how bright or warm a room seems.

For control, these processes must be expressed as measurable variables. Soil moisture becomes VWC, light becomes PPFD or lux, and temperature and humidity define the atmospheric load on the plant. Only numerical values allow operating ranges and thresholds to be defined and monitored. However, simply tracking numbers is not enough. To design an effective warning system, one must understand what happens when these boundaries are breached.

2.2.2 Effects of Environmental Factors on Plant Growth

Deficiency: when values are too low

Low soil moisture reduces water availability until the permanent wilting point is reached. Monitoring must warn before the plant approaches this boundary, because recovery becomes unreliable once accessible water is depleted. [?]

Low light creates a different stress pattern. The plant tends to grow taller and thinner as it stretches toward available light, often producing weaker stems and paler leaves. This gradual change indicates chronic under-lighting and is not resolved by a single bright day. [?]

Low temperature slows growth by reducing reaction rates. Low humidity increases transpiration demand, which means the plant can become water-stressed even when the soil appears adequately moist. [?]

Excess: when values are too high

Excess soil moisture displaces air in pores, starving roots of oxygen. The plant can then show wilting symptoms even in wet soil, because damaged roots cannot absorb water properly. This is why overwatering often looks like underwatering to the untrained eye. Prolonged wet conditions also favor root decay pathogens, accelerating damage further. For engineering control, the upper moisture limit is a safety boundary that must be enforced with alerts. [?, ?]

Excess light and heat can cause leaf scorch, especially when a shade-adapted plant is suddenly exposed to strong sun. The damage occurs during short peak exposures rather than from long-term averages, which means monitoring must capture peaks, not only means. [?]

Safe corridors and thresholds

A monitoring system needs a control model that prevents both deficiency and excess. The simplest model is a safe corridor: a defined acceptable range for each variable. Inside the corridor, stress risk is low. Near corridor edges, the system should warn before the plant enters a damage trajectory. These corridors are species-dependent, so they must be configurable rather than hard-coded. [?]

A practical example: many houseplants prefer relative humidity around 40–60%, while tropical species often prefer higher levels. This is exactly why a system should start with reasonable defaults and then refine them using observed trends. Data makes the corridor adaptive instead of theoretical.

2.2.3 Why Sensor-Based Logging Matters

Human perception is qualitative, biased, and slow compared to plant dynamics. Surface touch checks miss root-zone conditions, the eye adapts to darkness and overestimates available light, and weekly observation misses hourly stress peaks. [?, ?] Continuous logging at fixed intervals is not a luxury but a practical requirement, because it reveals patterns and variability that single observations cannot capture. [?]

Sensors convert hidden processes into objective numbers and timestamps. They enable threshold checks, trend detection, and early warnings before visible symptoms appear. This is the real value of Plant Up!: plant care becomes a monitored system, not a guessing game.

With the biological variables defined and the necessity of objective measurement established, the next chapter evaluates the physical hardware required and describes how sensor signals are captured, filtered, and transmitted as structured data.

2.3 Sensor Technologies and the Measurement Pipeline

Every environmental variable, whether soil moisture, temperature, humidity, or light, must be converted into an electrical signal before a controller can process it. The choice of sensor determines not only what can be measured, but also how accurately and how long the system can operate without maintenance. But selecting appropriate hardware is only the first step. The raw signals must also be read, filtered, and packaged into structured data before they can be evaluated. This chapter covers both: the sensor selection for Plant Up! and the firmware-level data pipeline that turns their output into usable information.

2.3.1 Overview of Common Sensor Types

Soil moisture sensors

Low-cost soil moisture measurement is typically implemented using resistive or capacitive sensors.

A resistive sensor works by passing a small electric current between two metal probes inserted into the soil. Wet soil conducts electricity better than dry soil, so the measured resistance drops as moisture increases. The principle is straightforward and the hardware is cheap, but there is a significant disadvantage: the metal probes are directly exposed to wet soil, which causes them to corrode over time. As the probes degrade, readings become increasingly unreliable. [?]

A capacitive sensor takes a different approach. Instead of measuring how well the soil conducts current, it detects how the surrounding material affects an electric field generated by the sensor. In simple terms, water changes the electrical properties of the soil around the sensor, and this change is what gets measured. The key advantage is structural: the sensing element is embedded inside a coated circuit board, so no metal is directly exposed to the soil. This eliminates the corrosion problem and makes the sensor significantly more durable for long-term use. [?]

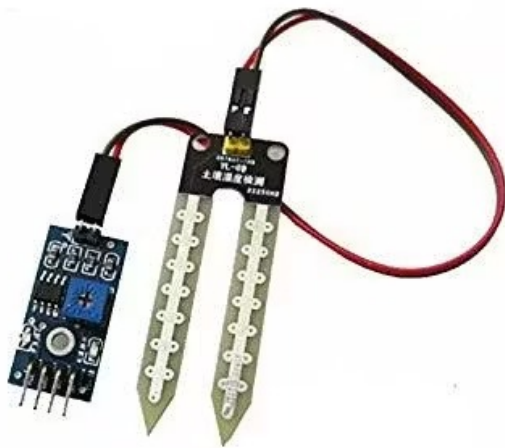


Figure 2.1: Resistive Soil Moisture Sensor [?]



Figure 2.2: Capacitive Soil Moisture Sensor [?]

As shown in Figures ?? and ??, the structural difference is immediately visible. The resistive sensor exposes two metallic forks directly to the soil, while the capacitive sensor uses a sealed PCB surface. This is the core reason why capacitive sensors are generally more suitable for long-term indoor monitoring. [?, ?]

Temperature and humidity sensors

Indoor monitoring often uses combined digital sensors such as the BME280. A digital sensor converts the measured value internally and sends a numerical result to the microcontroller, which eliminates the need to interpret raw analog voltages.

The BME280 communicates via I²C, a two-wire protocol that uses one data line and one clock line. Multiple sensors can share the same two wires, which simplifies wiring considerably. For indoor plant monitoring, where detecting trends and crossing thresholds matters more than achieving laboratory-grade accuracy, the BME280 provides sufficient precision. [?, ?]

Light sensors

Light intensity can be measured using a photoresistor or a digital lux sensor. A photoresistor changes its resistance depending on brightness. It is simple and inexpensive but mainly useful for relative comparisons within the same setup. A digital lux sensor, by contrast, outputs illuminance directly in calibrated lux values. [?]

As discussed in the previous chapter, plants respond to light in the PAR range (400–700 nm), and lux is weighted for human vision rather than plant needs. [?] For basic indoor monitoring, however, lux values are still practical enough to detect whether a plant is receiving adequate or insufficient light.

All three sensor categories, moisture, climate, and light, offer viable solutions for low-cost monitoring. However, selecting the right sensor type is only half the equation. Understanding their practical limitations is equally important, because no sensor delivers perfect data under real-world conditions.

2.3.2 Limitations and Typical Error Sources

Non-uniform soil moisture distribution

Soil inside a pot does not dry evenly. Water moves downward due to gravity, and localized wet zones can persist long after watering.

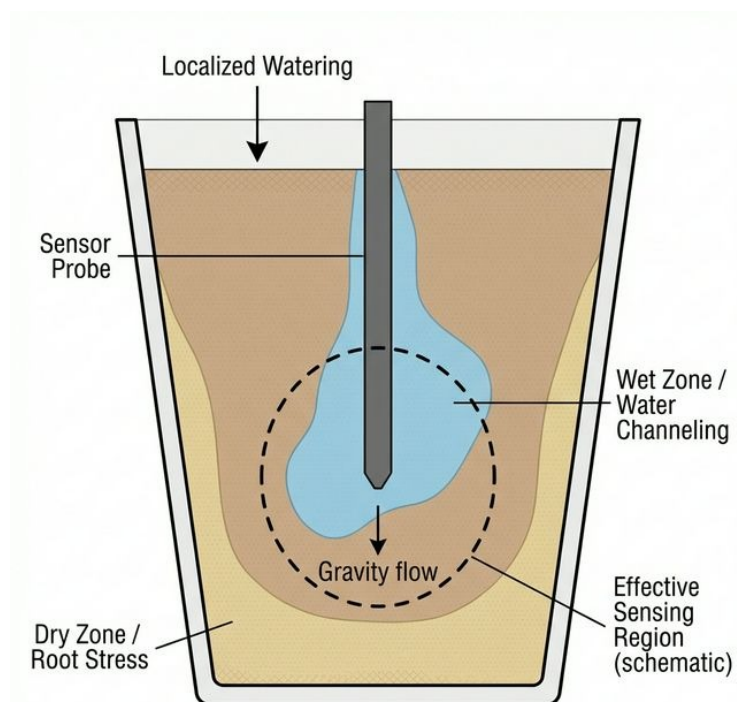


Figure 2.3: Measurement distortion caused by non-uniform soil moisture in a plant pot. The image illustrates how localized watering can distort sensor readings (AI-generated).

As shown in Figure ??, a soil moisture probe only measures a limited region around its body. A single reading may therefore not represent the entire pot. If the sensor happens to sit in a locally wet or dry pocket, threshold-based decisions can become misleading. Consistent sensor placement and insertion depth are therefore essential for reliable monitoring.

Sensor drift

Sensor drift describes a gradual shift in readings over time without any real change in the environment. For resistive sensors, corrosion is the primary driver: as the metal probes

degrade, the baseline resistance changes. Capacitive sensors reduce this problem significantly but can still drift due to aging or surface contamination. Periodic validation against a known reference improves long-term reliability. [?, ?]

Electrical noise and ADC limitations

Electrical noise refers to unwanted fluctuations in a signal caused by interference or unstable power supply. Analog sensors require an ADC (Analog-to-Digital Converter), which translates a continuous voltage into a discrete digital value. Because the resolution of the ADC is finite, very small moisture changes may not register at all. Hardware quality and basic software filtering, such as averaging multiple readings, therefore directly influence measurement stability.

2.3.3 Selection Criteria

For Plant Up!, a capacitive soil moisture sensor is the preferred choice. It avoids exposed metal electrodes, which eliminates the corrosion failure mode that limits resistive alternatives. This directly improves long-term stability, which is critical for a system intended to run continuously in a home environment. [?]

Compatibility with the ESP32 microcontroller is another practical constraint. The ESP32 operates at 3.3 V logic levels, so all selected sensors must support this voltage natively to avoid the need for additional level-shifting circuitry.

In this project, cost and durability outweigh the need for absolute calibration accuracy. The goal is reliable threshold monitoring, not scientific soil analysis. Knowing whether moisture is “inside the safe corridor” or “approaching the danger zone” matters more than knowing the exact volumetric water content to two decimal places.

Digital sensors such as the BME280 further simplify the hardware design. By communicating over I²C, they reduce analog noise issues and allow multiple sensors to share a single data bus. [?]

Based on these criteria, the implemented hardware setup of Plant Up! consists of an ESP32 microcontroller, a capacitive soil moisture probe, and a BH1750 digital light sensor. The ESP32 serves as the central processing unit and provides integrated Wi-Fi connectivity as well as 12-bit ADC channels for analog measurements. Its 3.3 V logic level defines the voltage compatibility requirements for all connected sensors.

The BH1750 light sensor communicates via I²C and directly outputs digital illuminance values in lux within a measurement range of approximately 1 to 65,000 lux. This avoids additional analog signal conditioning and simplifies integration.

The selected soil moisture probe is designed for 3.3 V operation and provides an analog voltage proportional to relative substrate moisture. Together, these components form the minimal stable sensing architecture of the system. A detailed component list with

supplier information is provided separately.

2.3.4 The Measurement Cycle

With the hardware selected, the remaining question is purely procedural: how does the controller read, clean, and transmit sensor values in a repeatable cycle?

Firmware read cycle

The firmware runs a repeating loop: wake, read sensors, process values, transmit, and sleep. Deep sleep, a power-saving mode where the CPU and most peripherals are shut down and only a small timer domain stays active for wake-up, is used after each transmission. [?]

The read sequence matters. The soil moisture sensor is analog, meaning its raw voltage needs a brief moment to stabilize after the controller wakes up. That is why it is read first. The BME280 and BH1750, on the other hand, are digital and deliver ready-to-use values over I²C as soon as they are asked. Reading analog first and digital second keeps the cycle fast and stable. The entire active phase stays below approximately 2 seconds, after which the controller goes back to sleep and waits for the timer to start the next round. [?]

Sampling strategy and power management

In Plant Up!, the default measurement interval is 10 minutes, though it can be configured. This is long enough to keep battery consumption low but short enough to capture typical indoor changes such as watering events, sunlight shifts, or heating cycles.

The design relies on the extreme difference between active and sleep current. During active operation with Wi-Fi enabled, the ESP32 draws around 240 mA and can briefly peak above 300 mA during wireless calibration. [?] In deep sleep, current consumption drops to approximately 10 μ A. [?] With timer-based automatic wake-up, this duty-cycling enables an estimated battery life of several weeks on a standard Li-Ion cell, depending on board design and signal quality. Shorter intervals, such as every 1–5 minutes, are supported for debugging and calibration but reduce runtime proportionally, because the Wi-Fi startup and transmission overhead becomes the dominant consumer. [?]

Signal processing and data cleaning

Each cycle takes multiple readings per sensor, typically five, and reduces them to one stable value. Simple averaging smooths random noise, while a median filter is useful when single spikes occur, for example from one bad ADC sample. This is especially

relevant for the analog soil moisture channel, where ADC fluctuations were already discussed in Section 2.3.2.

The firmware also performs plausibility checks. Physically impossible values are discarded, such as negative humidity or moisture readings above 100%. If a digital sensor does not respond within a timeout, the system logs a null value for that field and continues the cycle with the remaining sensors instead of blocking entirely. Null values are preserved in the payload so the backend can detect and handle data gaps.

Data packaging and transmission

After cleaning, values are assembled into a single JSON payload. JSON (JavaScript Object Notation) is a lightweight text format for structured data, built from name-value pairs. [?] A typical payload contains the device ID, a timestamp, and the current readings: soil moisture (%), temperature (°C), humidity (%), and light intensity (lux).

The payload is transmitted via MQTT, a lightweight publish/subscribe messaging protocol designed for constrained IoT devices. [?] How MQTT compares to REST in terms of latency and reliability is covered separately by Richard Onuoha in his chapter on IoT data synchronization in microservice architectures.

To keep the device robust in real apartments, where Wi-Fi can drop out or routers restart, the firmware includes retry logic. If the connection to the broker fails, the data point is buffered locally and retransmitted on the next cycle. No long-term storage is kept on the device. The intention is short buffering for connectivity gaps, not acting as a local database.

With this, the measurement cycle ends with clean, timestamped, structured values arriving at the backend. This is exactly the input needed for the next chapter, where thresholds and interpretation logic turn the raw data stream into actionable care recommendations.

2.4 Data-Driven Decision Making in Plant Care

After the measurement cycle described in Section 2.3, Plant Up! receives structured sensor data at the backend: timestamped values for moisture, climate, and light. This chapter explains how those incoming values are interpreted and transformed into feedback that a user can act on without needing to read raw numbers.

2.4.1 Threshold Zones and Plant-Specific Ranges

Building on the safe corridors defined in Section 2.2, each incoming sensor value is compared against an ideal range stored as a plant-specific configuration. The system classifies every parameter into three condition zones: Green (optimal, no action), Yellow

(close to a boundary, attention recommended), and Red (outside the safe range, action required).

This applies to soil moisture, temperature, humidity, light, and, if available, the electrical conductivity (EC) of the nutrient solution. The traffic-light mapping is intentionally chosen because green, amber, and red are widely recognized as low, medium, and high status codes in everyday decision aids.

Plant-specific ranges are not a universal default. A cactus is typically configured with lower moisture tolerance and higher light tolerance, while a fern prefers higher humidity and lower exposure to direct sunlight. In Plant Up!, these differences are handled as configurable parameters per plant profile rather than hardcoded constants.

Seasonal adjustment is implemented by storing separate summer and winter ranges. The reasoning is practical: indoor climate and light availability shift with seasons, and the same plant often needs different watering frequency and different tolerance bands depending on these conditions. Instead of forcing users to retune thresholds manually, Plant Up! switches between two predefined sets while keeping them editable.

To avoid overwhelming users with five separate decisions, Plant Up! computes a health score from 0–100% as a combined indicator. Such composite scoring simplifies interpretation without discarding the underlying data from individual sensors. [?] The health score compresses complexity into one overall status that is easy to scan. It is mapped to four labels: Great ($\geq 70\%$), Okay (40–70%), Needs Care (20–40%), and Critical ($< 20\%$).

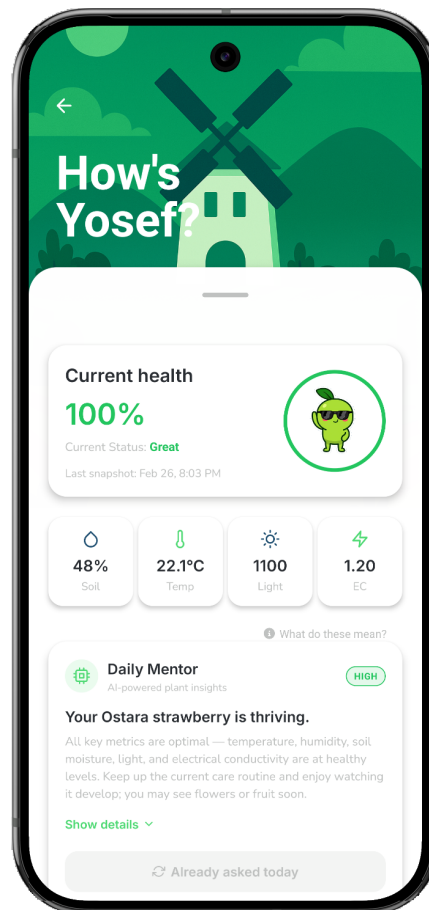


Figure 2.4: Plant status overview in the Plant Up! application showing health score, individual sensor values, and AI-generated care recommendations via the Daily Mentor feature.

As shown in Figure ??, the interface exposes both layers at once: the overall health score for quick orientation and the individual sensor values for users who want to verify the cause of a warning. The Daily Mentor feature is an AI-supported component that generates short natural-language hints based on the current zone states, turning combinations like “too dry” and “high temperature” into a concrete watering suggestion.

2.4.2 Deriving Care Recommendations

When a value leaves its ideal range, Plant Up! generates a recommendation phrased as an action, not as a measurement. Translating raw sensor readings into specific, user-facing actions is a well-established approach in threshold-based decision support systems. [?] The goal is that a user reads “Water it now” instead of interpreting “Soil moisture: 18%” under time pressure. The system evaluates parameters individually, and the first parameter that triggers a Yellow or Red state becomes the active recommendation until the next measurement cycle updates the situation.

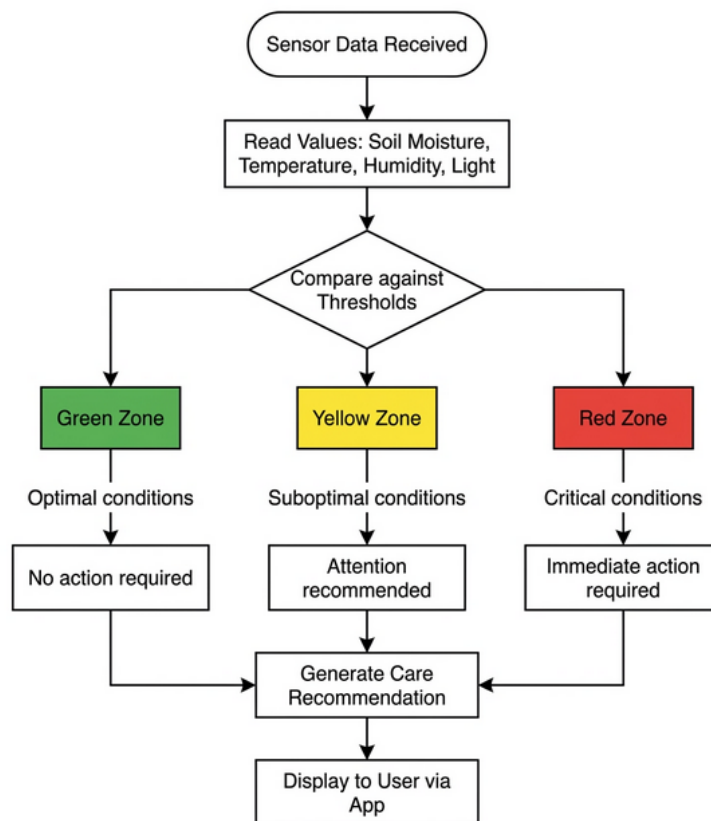


Figure 2.5: Simplified decision flow: sensor data is compared against plant-specific thresholds and classified into condition zones, which then trigger the corresponding care recommendation (AI-generated).

Figure ?? summarizes this logic visually: incoming measurements are checked against stored thresholds, classified into zones, and then translated into one user-facing recommendation.

2.4.3 Visualization and Interpretation

Interpretation is not only logic. It is also presentation. Plant Up! uses a traffic-light visualization with color-coded indicators (green, yellow, red) and an animated mascot (“Sprig”) that mirrors the current plant state. The purpose is speed: users should recognize “all good” versus “act now” in one glance, even without understanding the sensor units. Using color as a semantic status channel follows the principle that red signals danger, yellow signals caution, and green signals a safe condition, a convention standardized in ISO 3864. [?]

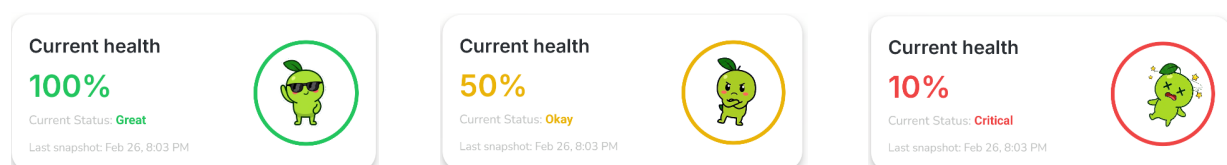


Figure 2.6: Traffic light health status in Plant Up!: optimal conditions (left, 100%), suboptimal conditions requiring attention (center, 50%), and critical state requiring immediate action (right, 10%).

Figure ?? illustrates these three primary health states. Rather than focusing on numbers, the interface leverages the mascot's expression and the background color to immediately communicate whether the plant is thriving, needs attention, or requires urgent care.

Single values are useful, but trends are where the “why” becomes visible. Plant Up! therefore shows historical data as bar charts with selectable time ranges (1 hour, 24 hours, 7 days, all time). Time-oriented visualizations make it easier to detect patterns and anomalies than isolated snapshots, because humans can quickly see slopes, cycles, and sudden jumps across a timeline. [?]

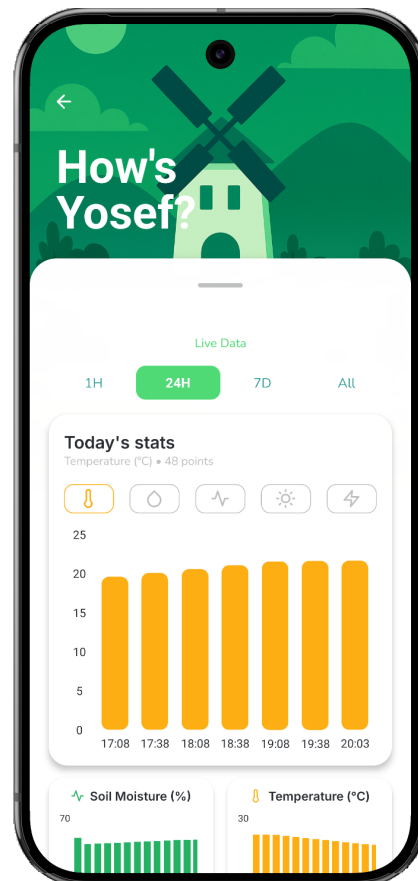


Figure 2.7: Historical sensor data visualization in Plant Up! showing temperature readings over a 24-hour period. Users can switch between different sensors and time ranges to identify trends and patterns.

As illustrated in Figure ??, a 24-hour chart can reveal repeating daily peaks, such as afternoon heating near a window, or unusual drops from night ventilation. These patterns support long-term care decisions like relocating the plant or adjusting the watering schedule proactively.

With these decision rules and visual mappings defined, Plant Up! can transform raw measurements into consistent feedback. The next chapter describes how this interpretation layer is implemented in the real system and tested under practical conditions.

2.5 System Implementation and Validation

This chapter describes the practical realization of the Plant Up! sensor station. With the sensor selection (Section 2.3.3), firmware pipeline (Section 2.3.4), and decision logic (Chapter 2.4) already established, the focus here is on three things that have not yet been covered: how the components were physically wired and housed, how raw ADC readings were calibrated into meaningful moisture values, and how the assembled system was validated against real plant data.

2.5.1 Hardware Assembly

- **Wiring and pin assignment:**
 - GPIO pin mapping table: which ESP32 pin connects to which sensor
 - Wiring diagram or schematic as figure
- **Enclosure:**
 - Custom 3D-printed housing for indoor use
 - Protection against water splashes, ventilation openings for climate sensor
 - Photo of the finished sensor station as figure

2.5.2 Sensor Calibration

- **Problem:**
 - ESP32 ADC produces raw values (0–4095) with no inherent unit
 - A user cannot interpret “ADC reads 2340” without context
- **Two-point calibration:**
 - Step 1: Record raw value in air-dry substrate (dry reference)
 - Step 2: Record raw value in fully saturated substrate (wet reference)
 - Linear mapping between these two endpoints to a 0–100% moisture scale
 - Concrete calibration values from the Plant Up! test setup
- **Limitations:**
 - Calibration is substrate-specific (different soil types produce different base-lines)
 - Plant Up! provides relative moisture indication, not laboratory-grade VWC
 - Recalibration recommended when substrate or pot changes

2.5.3 Practical Validation

- **Test setup:**
 - Specific plant type, location, and monitoring duration
 - Controlled watering event during test phase
- **Expected vs. measured behavior:**
 - Soil moisture: spike after watering, gradual decline from evaporation and uptake (consistent with theory from Section 2.2)
 - Light: daily cycle visible (bright daytime, low at night)
 - Temperature and humidity: stable baseline with minor shifts from heating or ventilation
- **Evidence:**
 - At least one figure showing real sensor data from the Plant Up! app
 - Visual comparison between measured curve and expected physical behavior
- **Limitations:**
 - Limited number of test plants and short test duration
 - No laboratory reference sensor for absolute accuracy verification
 - Results validate functional correctness, not scientific precision

2.6 Conclusion

This thesis set out to answer a practical question: how can a low-cost, sensor-based system provide objective environmental data and actionable care recommendations for indoor plants? The chapters above have addressed this question step by step, from the biological foundations through sensor selection and firmware design to the interpretation logic that turns raw measurements into user-facing feedback.

The core argument of this work is that plant care does not need to rely on intuition. Soil moisture, temperature, humidity, and light are physical quantities with measurable ranges, and when those ranges are violated, the consequences are predictable. Chapter 2.2 established that both deficiency and excess cause specific, well-documented stress responses, and that human perception alone is too slow and too biased to detect these conditions reliably. Chapter 2.3 then showed that affordable consumer-grade hardware, specifically a capacitive soil moisture sensor, a BME280 climate sensor, and a BH1750 light sensor connected to an ESP32 microcontroller, can capture these variables with sufficient accuracy for threshold-based monitoring. The firmware pipeline described in Section 2.3.4 ensures that sensor data is read, filtered, and transmitted in a repeatable cycle with minimal power consumption.

Chapter 2.4 translated these raw measurements into decisions. By classifying each parameter into green, yellow, and red zones and combining them into a single health score, Plant Up! compresses complexity into feedback that requires no botanical knowledge to understand. The traffic-light system, the animated mascot, and the trend visualizations all serve the same purpose: making the plant's actual state visible at a glance.

The system is not perfect. The sensor station measures a single point in the pot, not the entire root zone. Calibration is substrate-dependent and produces relative values rather than absolute ones. The health score is a weighted approximation, not a scientific diagnosis. These are deliberate trade-offs. The goal was never laboratory-grade precision but a practical tool that prevents the most common care mistakes, especially overwatering, which remains the leading cause of houseplant failure.

By relying entirely on widely available consumer-grade components and open protocols, the system confirms the premise from the introduction: effective plant monitoring does not require specialized equipment or expert knowledge. All data remains under user control, and the architecture supports scaling to multiple plants through individual sensor stations reporting to a shared backend.

With the sensor station producing clean, timestamped measurements and the interpretation logic translating them into actionable feedback, the next question becomes: where should this data be processed, and how does it reach the backend? The following chapter by Abd-Ulrahman Behari addresses exactly this, comparing edge and cloud processing strategies for environmental sensor data in the context of smart gardening.

Kapitel 3

Edge vs. Cloud Processing in Smart Gardening: A Comparative Study for Environmental Sensor Data

Author: Abd-Ulrahman Behari

3.1 Introduction

3.1.1 Background

Microservices in Gardening

- Paradigm shift, individual plant care, community interaction, shared data, digital transformation, nature meets tech, social platforms, sensor integration, remote monitoring, enthusiast community, collaborative gardening, data-driven plant care, IoT connectivity
- **[INSERT FIGURE: Concept Art of Social Gardening with IoT]**

The Latency Challenge: Real-time Notification vs. Cloud Round-Trips

- Technical hurdles, delay management, sensor reading, user notification delay, round-trip time (RTT), network jitter, cloud infrastructure, internet dependency, timely intervention, timing criticality, processing lag, signal propagation, feedback loops
- **[INSERT DIAGRAM: Round-Trip Time Visualization (Sensor → Cloud → Push Notification)]**

Dependency Risks: What happens to the plant when the Microservice is down?

- Centralized architecture, single point of failure, microservice downtime, plant health risks, missed alerts, service outages, server crashes, connectivity loss, maintenance windows, failover mechanisms, contingency planning, disaster recovery

3.1.2 Research Question

Primary Question

- **"What are the trade-offs between edge and cloud processing for real-time plant monitoring in consumer gardening systems?"**
- Edge computing benefits, Cloud computing benefits, latency vs. throughput, cost vs. complexity, data integrity, system reliability, user experience impact

3.1.3 Scope and Limitations

Focus on Consumer Hardware (ESP32/ESP8266)

- Consumer-grade electronics, ESP32 capabilities, ESP8266 limitations, cost-effectiveness, accessibility, hobbyist market, maker community, hardware constraints, memory limits, processing power, WiFi modules, microcontroller adoption
- **[INSERT PICTURE: ESP32/ESP8266 Microcontroller Setup]**

Comparison limited to Processing Location (Edge vs. Cloud), not Transport Protocols (MQTT assumed)

- Scope boundary, processing location focus, edge computing, cloud computing, transport layer abstraction, MQTT protocol, standard compliance, exclusion of CoAP/HTTP, architectural comparison, logic placement, fixed communication variables

3.2 Theoretical Framework

3.2.1 Modern IoT Backends

Backend-as-a-Service (BaaS): The Role of Supabase in IoT

- Backend-as-a-Service, BaaS, Supabase architecture, real-time subscriptions, web-sockets, rapid development, managed infrastructure, scalability, postgres-native, authentication, database APIs, serverless paradigm, unified backend
- **[INSERT DIAGRAM: Supabase Realtime Architecture Flow]**
- **Source:** <https://metadesignsolutions.com/supabase-vs-firebase-when-to-choose#:~:text=Supabase%20is%20an%20open%2Dsource, power%20of%20a%20relational%20database.>

Time-Series Databases: Why standard SQL fails for sensor streams

- Relational database limitations, high-velocity ingestion, storage bloat, index inefficiency, B-tree performance, standard SQL constraints, sensor data volume, continuous streams, data retention issues, query degradation
- **Source:** <https://milvus.io/ai-quick-reference/what-are-the-limitations-of-relational-databases-for-time-series-data.html>

The TimescaleDB Extension: Hypertables and Chunking explained

- PostgreSQL extension, TimescaleDB, hypertables, automatic partitioning, data chunking, temporal locality, high-performance inserts, time-series optimization, compression algorithms, continuous aggregates, SQL compatibility, retention policies
- **[INSERT DIAGRAM: Hypertable vs. Normal Table Storage]**
- **Source:** <https://www.tigerdata.com/learn/the-best-time-series-databases-for-2023/>

Object Storage and Media Handling in Microservices

- Unstructured data storage, Supabase Storage, S3 compatibility, scalable object storage, media asset management, separation of concerns (DB vs. Storage), CDN caching strategies, secure file access policies, handling large binary files (blobs)
- **Source:** <https://github.com/supabase/storage>

3.2.2 Microservices in IoT

The "Gateway Pattern: Using a C# Service to bridge MQTT and Database

- Gateway design pattern, architectural patterns, C# middleware, .NET Core, MQTT broker, database bridge, decoupling services, protocol translation, message buffering, security boundary, data normalization, traffic throttling
- **[CODE SNIPPET: C# Worker Service Basic Structure]**
- Source: <https://microservices.io/patterns/apigateway.html>

Service Inter-communication: Synchronous vs. Asynchronous patterns

- Inter-service communication, synchronous REST, asynchronous messaging, message queues, event-driven architecture, request-response, immediate consistency, eventual consistency, system coupling, latency impact, blocking operations
- Source: <https://www.geeksforgeeks.org/system-design/microservices-communication-patterns/>

3.2.3 Edge Computing Patterns

Cloud Computing, Fog Computing, and Edge Computing

- **Cloud Computing:** Centralized processing, high computational power, high latency, infinite storage, global access.
- **Fog Computing:** Intermediate layer, gateway processing, local area network, bridging Edge and Cloud, reduced latency compared to Cloud.
- **Edge Computing:** Source processing, on-device logic, microcontroller level, lowest latency, real-time response, limited resources.
- **[INSERT FIGURE: Cloud vs. Fog vs. Edge Architecture Pyramid]**

Thick Edge"vs. Thin Edge: How much logic should the microcontroller hold?

- Edge architecture, Thick Edge, Thin Edge, logic distribution, processing offload, local autonomy, bandwidth conservation, resource constraints, computational power, firmware complexity, distributed intelligence, architectural trade-offs, remote management

3.3 The PlantUp Architecture

3.3.1 System Overview

High-Level Diagram: From Soil Sensor to React Native App

- Full-stack architecture, system topology, soil moisture sensors, analog-to-digital conversion, capacitive sensing, React Native framework, mobile application, user interface, cloud synchronization, end-to-end data flow, interactive dashboard
- **[INSERT FIGURE: High-Level System Architecture Diagram]**
- **Source:** <https://www.geeksforgeeks.org/postgresql/postgresql-schema/>
<https://www.geeksforgeeks.org/system-design/architecture-of-a-system/>

3.3.2 The Microservices Layer (Cloud)

User & Social Media Services: Gamification and Content Sharing

- Social microservices, gamification engine, PostgreSQL schemas, user profiles, experience points (XP), leaderboards, achievement system, social graphs, user interactions, relational data modeling, reward logic, competition mechanics
- **Social Media Features:** Video upload pipeline, automatic thumbnail generation, post creation, Supabase Storage buckets, media metadata, content delivery network (CDN) integration, secure signed URLs, social feed algorithms
- **[INSERT DIAGRAM: Gamification Database Schema (ERD)]**
- **[INSERT DIAGRAM: Video Upload and Thumbnail Generation Flow]**
- **Source:** <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>
<https://supabase.com/docs/guides/storage/cdn/fundamentals>

Webshop Service: Shopify Integration

- Shopify platform, SaaS e-commerce, API integration, inventory management, external service, seamless user experience, checkout process, webhook sync, product catalog, managed hosting, third-party plugin ecosystem
- **[INSERT SCREENSHOT: PlantUp Shop Concept on Shopify]**
- **Source:** <https://www.cloudflare.com/de-de/learning/cloud/what-is-saas/>
<https://www.techtarget.com/searchcloudcomputing/definition/Software-as-a-Service>

Microcontroller Management Service: The C# .NET Core Worker

- Background worker, .NET ecosystem, device management, command orchestration, state synchronization, keep-alive monitoring, message processing, centralized control, device registry, firmware updates (OTA), scalable architecture
- **[CODE SNIPPET: C# Device State Manager Class]**
- Source: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/host/hosted-services?view=aspnetcore-10.0&tabs=visual-studio>

3.3.3 The Data Persistence Layer (Supabase)

Unified Database Strategy: Single Postgres instance with logical schema separation

- Database consolidation, single instance, logical isolation, schema-based multi-tenancy, operational simplicity, reduced overhead, unified backup, cross-schema querying, management efficiency, Postgres roles, access control

The microcontroller_data Schema: Utilizing the TimescaleDB Extension

- Dedicated schema, TimescaleDB integration, sensor data storage, time-series optimization, optimized querying, high-volume tables, data partitioning, specialized indexing, analytical queries, storage efficiency, historical data
- **[CODE SNIPPET: SQL Creation Script for TimescaleDB Hypertable]**
- Source: <https://www.influxdata.com/time-series-database/>
<https://github.com/timescale/timescaledb>

Hypertable Design: Partitioning strategy for millions of sensor readings

- Hypertable architecture, data partitioning, time intervals, chunk management, scalable storage, query performance, massive datasets, millions of rows, retention management, automated maintenance, disk I/O optimization
- **[INSERT DIAGRAM: Visualizing Hypertable Chunks based on Time]**
- Source: <https://www.cloudthat.com/resources/blog/scaling-time-series-dat>

3.4 Methodology

3.4.1 Experimental Architectures

Architecture A: Cloud-Centric (The ConnectedPPlant)

- Cloud-centric design, connected model, centralized decision making, MQTT transmission, C# service logic, historical querying, time-weighted averages, notification latency, server-side processing, dumb terminal, remote monitoring, bandwidth dependence
- **[INSERT DIAGRAM: Architecture A Data Flow (Sensor → Cloud → Push Notification)]**
- Source: <https://www.ibtimes.com/architecting-connected-retail-future-cl>

Architecture B: Edge-Centric (The SSmartPPlant)

- Edge-centric design, smart device, distributed intelligence, local processing, independent operation, C++ logic, asynchronous logging, eventual consistency, disconnect tolerance, local status indication, autonomous state monitoring, bandwidth efficiency
- **[INSERT DIAGRAM: Architecture B Data Flow (Sensor → Edge → Local Indication/Async Log)]**
- Source: <https://www.mdpi.com/1424-8220/20/3/892>

3.4.2 Implementation Details

Edge Firmware (C++): Sensor Calibration and Power Management

- Firmware development, C++ programming, embedded systems, capacitive moisture sensing, signal smoothing, debouncing, deep sleep modes, battery optimization, data packetization, interrupt handling, robust error recovery
- **[CODE SNIPPET: C++ ESP32 Deep Sleep & Sensor Reading]**

Cloud Service (C#): Implementation of the Supabase.Client and Supabase.Realtime listeners

- Cloud service implementation, C# coding, Supabase SDK, real-time listeners, websocket subscription, event handling, state synchronization, database triggers, remote procedure calls, responsive backend, client library integration

- **[CODE SNIPPET: C# Supabase Realtime Listener Setup]**

- Source: <https://github.com/supabase/realtime>

Database Schema: Definition of sensor_readings hypertable

- Schema definition, DDL statements, hypertable creation, column types, indexing strategy, partition keys, compression settings, data persistence, storage optimization, analytical capabilities, query planning
- **[CODE SNIPPET: SQL for Hypertable Creation and Compression Policy]**
- Source: <https://gunesramazan.medium.com/managing-iot-heartbeat-data-with>

Power Supply Design and Consumption Analysis

- **Worst Case Scenario:** Continuous operation, all sensors active, WiFi transmission 24/7, 168 mA continuous draw, 4032 mAh daily consumption, theoretical maximum, stress testing benchmarks
- **Realistic Strategy (Deep Sleep):** Cyclic operation, 10-minute reporting interval, 2-5 mA average consumption, user-configurable frequency, deep sleep (0.1 mA), active burst (150 mA for 3s), efficient power management
- **Power Architecture:** Rechargeable LiPo battery (3.7V), Dual charging path (USB-C + Solar), BQ24074 Power Management Module, automatic source switching, stable voltage regulation, outdoor autonomy
- **Component Specifications:** LiPo Capacity (3000-5000 mAh), Solar Panel (5-6V, 1-2W, outdoor-proof), Voltage Regulator (Integrated)
- **Battery Life Projections:**
 - 3000 mAh Battery: 3 mA avg → 1000 hours → approx. 41 days
 - 5000 mAh Battery: 3 mA avg → 1666 hours → approx. 69 days (>2 months)
- **[INSERT GRAPH: Power Consumption Profile (Deep Sleep vs Active Burst)]**
- Source: <https://running-system.atlassian.net/wiki/spaces/RS/pages/126550051/Power+Management+Research>

3.4.3 Test Scenarios

- Source: <https://www.geeksforgeeks.org/software-testing/system-testing/>
<https://github.com/timescale/tsbs>
<https://www.bytedbase.com/blog/postgres-row-level-security-limitations>
<https://supabase.com/docs/guides/realtime/benchmarks>

Scenario 1: Ideal Conditions

- Baseline testing, ideal environment, low latency, zero packet loss, stable connectivity, theoretical maximums, benchmarking, optimal performance, clean signal, control group, reference measurements

Scenario 2: Database Load (Simulating 1,000 concurrent writes to TimescaleDB)

- Load testing, stress test, concurrent users, high ingestion rate, database bottlenecks, resource contention, write blocking, 1000 connections, performance degradation, scalability limits, system stability, latency spike
- **[INSERT PICTURE: Load Testing Setup / Console Logs]**

Scenario 3: Service Outage (Simulating a crash of the C# Microservice)

- Resilience testing, fault injection, service crash, chaos engineering, availability test, outage simulation, watchdog timers, system recovery, safety verification, data loss analysis, fail-safe behavior

3.5 Results and Data Analysis

3.5.1 Latency Analysis

Edge Response Time (<50ms) vs. Cloud Round Trip (>200ms + DB Query Time)

- Quantitative analysis, edge speed, sub-50ms response, cloud delay, >200ms latency, network overhead, database query time, round-trip metrics, comparison charts, performance penalty, real-time constraints, user experience impact
- **[INSERT GRAPH: Comparison Bar Chart of Alert Generation Time (Edge vs. Cloud)]**
- Source: <https://ifactoryapp.com/blog/real-time-ai-cloud-latency-manufact>

3.5.2 Database Performance

Impact of High Ingestion Rates on C# Service Query Performance

- Performance impact, ingestion throughput, query latency, lock contention, C# service load, resource usage, CPU/Memory metrics, read-write conflict, optimization

analysis, database tuning, concurrency issues

- **[INSERT GRAPH: Query Latency vs. Ingestion Rate]**

Storage Efficiency: Raw Data (Cloud Decision) vs. Batched Aggregates (Edge Decision)

- Stagemetrics, raw data volume, aggregation efficiency, payload size, storage costs, bandwidth consumption, cloud storage, batched transmission, data compression, economy of scale, archival strategy
- **[INSERT CHART: Projected Storage Costs Over Time]**
- Source: <https://metadeskglobal.com/from-raw-sensor-data-to-intelligent-a>
<https://stonefly.com/blog/iot-data-storage-architectures-databases/>

3.5.3 Reliability and Resilience

Alert Delivery Success Rates during a simulated 1-hour internet outage

- Reliability metrics, delivery success analysis, outage simulation, local data buffering, resync capability, data integrity, critical failure, connectivity loss, edge resilience, robust design, message queuing
- **[INSERT TIMELINE: Data Sync Recovery after 1-Hour Outage]**
- Source: <https://www.opal-rt.com/blog/5-essential-computer-simulation-tec>

3.5.4 Gamification Synchronization

The “Lag” between Action (Watering) and Reward (XP Update in Social Service)

- User experience, feedback delay, action-reward loop, gamification lag, sync latency, psychological impact, interface responsiveness, XP updates, social satisfaction, system consistency, near real-time
- **[INSERT DIAGRAM: Sequence Diagram of Action-to-Reward Delay]**
- Source: <https://supabase.com/docs/guides/functions>

3.6 Discussion

3.6.1 The "HybridSSweet Spot for PlantUp

Using Edge for Real-time Monitoring and Cloud for Engagement (XP/Leaderboards)

- Architectural synthesis, hybrid model, best of both worlds, edge reliability, cloud scalability, critical vs. non-critical, monitoring logic, engagement layer, strategic separation, optimal design, computing paradigm balance
- **[INSERT DIAGRAM: The Hybrid SSweet SpotLLogic Distribution]**
- Source: <https://ieeexplore.ieee.org/document/11158770>

3.6.2 Database Considerations

Benefits of Supabase's unified access for the React Native App

- Developer experience, unified API, frontend simplification, React Native integration, simplified auth, reduced complexity, rapid prototyping, coherent data model, streamlined access, architectural elegance
- Source: <https://taglineinfotech.com/blog/which-backend-is-best-for-react-native>

Trade-offs of querying TimescaleDB for real-time control loops

- Design trade-offs, real-time query risks, database dependency, latency variability, connection overhead, system stability, control loop jitter, architectural decisions, load implications, performance vs. consistency
- Source: <https://www.tigerdata.com/blog/timescaledb-vs-influxdb-for-time-series>

3.6.3 Complexity vs. Cost

Development cost of maintaining logic in two places (C++ and C#) vs. centralized C# logic

- Cost-benefit analysis, maintenance overhead, code duplication, technological diversity, C++ vs C#, staffing requirements, development velocity, operational complexity, total cost of ownership, robustness justification, long-term viability
- **[INSERT TABLE: Cost/Complexity Comparison Matrix]**

- **Source: Chapter 2.1** https://flame-challenge.authorea.com/users/982330/articles/1348311/master/file/data/wileyNJDv5_AMA/wileyNJDv5_AMA.pdf
<https://www.baeldung.com/cs/distributed-systems-thin-vs-thick-clients>
<https://www.einfochips.com/blog/understanding-risks-in-over-the-air-firmware-updates/>

3.7 Conclusion

3.7.1 Summary of Findings

- Research summary, key takeaways, latency findings, reliability confirmation, architectural verdict, performance benchmarks, hypothesis validation, data-driven conclusions, system evaluation, project goal achievement

3.7.2 Architectural Recommendations for PlantUp

Adopt Edge-First Monitoring to guarantee data integrity

- Strategic recommendation, Edge-First philosophy, integrity priority, decentralized monitoring, robustness first, fail-safe architecture, local buffering, connectivity independence, critical path optimization

Use Supabase Edge Functions (optional) vs. C# Service for lighter tasks

- Alternative architecture, serverless functions, Supabase Edge Functions, lightweight logic, cost reduction, operational simplicity, scaling flexibility, C# service replacement, modernization, reduced maintenance

3.7.3 Future Work (e.g., Predictive Health Alerts using Machine Learning on TimescaleDB data)

- Future directions, predictive analytics, machine learning, AI integration, smart health alerts, user guidance, data mining, TimescaleDB analytics, proactive care, intelligent systems, next-generation IoT
- **[INSERT CONCEPT ART: Future AI Dashboard]**

3.7.4 Connection to the Next Chapter

Since the entire communication architecture is based on MQTT to ensure real-time performance and reliability, the system avoids the complexities of hybrid protocol management. The following chapter details the trade-offs between MQTT and REST, and how the IoT Sync mechanism leverages this MQTT foundation to ensure seamless data synchronization.

Kapitel 4

IoT Data Sync in Microservices: Evaluating MQTT vs. REST

Author: Richard Onuha

4.1 Introduction

4.1.1 Background and Context

Over the past decade, the Internet of Things (IoT) has gradually moved from being a rather conceptual vision to a technology embedded in everyday life. Connected devices now monitor and influence physical environments in ways that might have seemed impractical only a few years ago. This evolution is particularly visible in agriculture. The concept of “Agriculture 4.0” effectively captures this transition from experience-based decision-making toward data-driven optimization, supported heavily by sensor networks and automated systems [?].

Within industrial contexts, these technologies have already led to measurable efficiency gains. Precision irrigation, automated fertilization, and large-scale environmental monitoring are no longer experimental ideas; they have become established practices. Around the same time, a comparable transformation has begun to surface on a much smaller scale in urban environments. Under the label of Smart Urban Gardening, digital tools are increasingly integrated to support plant cultivation in apartments, on balconies, and within community gardens [?].

From a broader perspective, this trend seems closely connected to ongoing societal developments. Rapid urbanization, reduced living space, and a growing collective awareness of sustainability have all contributed to a renewed interest in self-cultivation and local food production [?]. In parallel, the concept of the Social Internet of Things (SIoT) has emerged. This extends the traditional IoT model by allowing objects to form relationships, not only with other devices but actively with users [?]. Under such a framework, connected objects become part of a social ecosystem rather than functioning merely as isolated technical components [?].

When applied to plant care, this perspective implies that a plant, if supported by the right sensors and network connectivity, can start to communicate its condition. It is therefore no longer perceived solely as a passive object requiring observation, but rather as an entity whose specific needs are digitally represented and shared. While this idea initially appeared mostly conceptual, it undoubtedly has practical engineering consequences, particularly when attempting to implement it using affordable, consumer-grade hardware.

Unlike industrial systems that operate with highly stable power supplies and professionally managed infrastructure, consumer IoT devices generally must function under much tighter constraints [?]. Devices placed in residential homes often rely on limited battery power and are typically forced to operate within congested, sometimes highly unstable Wi-Fi networks. Under these circumstances, hardware selection, firmware design, and the choice of communication protocols must all be carefully considered and optimized [?]. Even small inefficiencies that might be entirely negligible in an industrial setting can quickly become critical flaws in battery-powered deployments.

4.1.2 Case Study: The “Plant Up!” Project

The present thesis investigates some of these specific challenges through the case study of the “Plant Up!” project. The system was originally conceived as an IoT-based application combining continuous plant monitoring with gamification elements, aiming to make routine plant care slightly more engaging [?]. Rather than simply displaying raw sensor values, the platform attempts to translate complex biological processes into more meaningful digital feedback for the user.

Its underlying architecture follows a fairly standard three-tier model, consisting of distributed sensor nodes, a scalable cloud-based backend, and a mobile application interface [?]. At the edge layer, ESP32-based devices continually measure environmental parameters such as soil moisture, ambient temperature, humidity, and light intensity. These measurements are then transmitted to a broader cloud infrastructure, where microservices gradually process the incoming data, apply relevant business logic, and update the persistent system state. Following this, the mobile application retrieves and visualizes the information, presenting it in a format intended to emphasize progress, interactive achievements, and overall plant well-being.

While this overall structure may appear reasonably straightforward, the actual implementation tends to reveal several architectural tensions. The continuous interaction between constrained edge devices and an expanding cloud backend requires rather careful coordination. It becomes evident that decisions made at the protocol level can directly and measurably influence system responsiveness, subsequent energy consumption, and the users’ perceived reliability of the platform.

4.1.3 Problem Statement

Designing a socially integrated, near real-time plant monitoring system inevitably exposes a fundamental trade-off between latency, energy efficiency, and data consistency. On one side of the equation, gamification mechanisms rely heavily on immediate feedback. When a user physically waters a plant, the expectation is usually that this action is reflected in the application almost instantly, without noticeable delay [?]. Delayed responses pose a significant risk of diminishing user engagement over time.

Nevertheless, battery-powered sensor nodes must aggressively minimize their energy consumption if they are to remain practical for everyday home use. The ESP32 platform, for instance, provides a Deep Sleep mode that significantly reduces current draw by disabling both the CPU and the Wi-Fi subsystem [?]. Waking from this state, however, requires full reinitialization and network reconnection, a process that inherently introduces both latency and additional energy expenditure [?].

This recurrent process leads to what can be described as a “wake-up tax”: before any actual, useful data can be transmitted, the device is forced to re-establish its network connectivity [?]. Interestingly, the specific communication protocol chosen for the actual telemetry transmission directly affects the magnitude of this unavoidable overhead. Traditional HTTP-based REST communication is widely supported and remains conceptually simple; yet, it involves relatively verbose textual headers and somewhat heavy message formats. MQTT, in contrast, was designed specifically with such constrained devices in mind. It utilizes a lightweight, binary protocol structure organized around a publish/subscribe model [?].

At this point, it is necessary to consider not only the immediate performance metrics but also overall system consistency. In distributed systems, the CAP theorem fundamentally states that it is impossible to guarantee consistency, availability, and partition tolerance simultaneously in the presence of network failures [?]. In standard residential IoT environments, temporary network partitions are certainly not exceptional events; they are common, almost expected occurrences. The architecture must therefore be designed to tolerate intermittent connectivity while still maintaining an acceptable, workable level of data correctness and service continuity.

Building upon these various considerations, the central research question of this thesis is formulated as follows:

How do MQTT and REST compare in a microservices-based architecture for real-time plant monitoring, specifically with respect to latency, throughput, and data consistency, when constrained by the energy limitations of battery-powered IoT devices?

To approach this question methodically, the following chapters first outline the most relevant theoretical foundations regarding modern communication protocols and distributed systems. The specific system architecture of the “Plant Up!” project is then examined in more detail, highlighting the underlying design decisions that guided its development. Finally, these architectural decisions are evaluated through a set of controlled experimental measurements in order to adequately assess their true practical implications.

4.2 Theoretical Background

Having broadly established the core challenges of latency, energy efficiency, and data consistency in the previous sections, it is helpful to outline the theoretical foundations necessary to fully grasp the architectural decisions made in this thesis. This exploration covers the shift toward microservices architecture, the somewhat harsh physical constraints of IoT hardware, the nuances of communication protocols, and overall distributed data consistency.

4.2.1 Microservices Architecture (MSA)

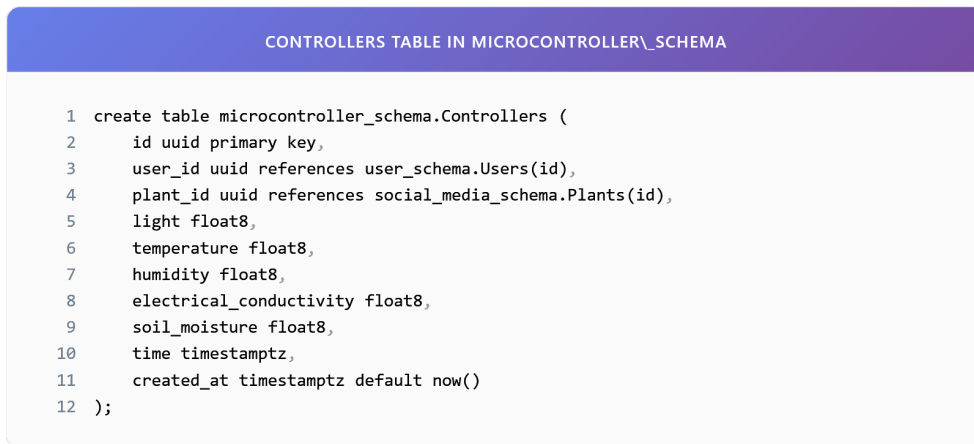
Microservices Architecture (MSA) represents a fairly fundamental shift in modern software design. It tends to organize applications not as massive, single monolithic systems, but rather as suites of small, relatively autonomous services that collaborate by communicating through well-defined interfaces [?]. In a traditional monolithic setup, almost all functionality, ranging from simple user management to heavy data processing, is tightly coupled within one large, interconnected codebase. While this approach admittedly simplifies initial development and deployment, it often morphs into a significant obstacle to scalability and fault tolerance as the application expands in complexity [?].

MSA conceptually addresses this limitation by treating the overarching application as a flexible collection of independent services. Each service generally runs in its own isolated process and communicates through lightweight channels, mostly using HTTP APIs or dedicated messaging buses [?]. Each distinct service takes responsibility for a specific business domain. For the “Plant Up!” platform in particular, this architectural style seemed well-suited. It allows completely distinct parts of the system to be developed, adjusted, and scaled independently over time. For instance, the specific service tasked with ingesting thousands of sensor readings per minute can theoretically be scaled horizontally during peak watering periods without inadvertently taxing the social feed service, which might be experiencing much lower, steady load at that very moment.

To manage this, the “Plant Up!” backend essentially organizes around domain-specific schemas within Supabase, which function as the dedicated data stores for these separate microservices:

- `user_schema`: This primarily manages user identity, safe authentication, and personal statistics like activity streaks.
- `social_media_schema`: Responsible for organizing the community features, generally including user posts and public plant profiles.
- `gamification`: Houses the somewhat complex logic for quest progression, experience points (XP), and virtual rewards.
- `microcontroller_schema`: Acts as a highly resilient data warehouse dedicated solely to the raw sensor telemetry arriving from the IoT edge nodes.

A remarkably clear example of this separation can be found in the actual storage of the sensor readings themselves. They reside in an entirely isolated table, structurally decoupled from the standard user data:



The image shows a SQL query in a code editor. The title bar of the editor reads "CONTROLLERS TABLE IN MICROCONTROLLER_SCHEMA". The query is as follows:

```
1 create table microcontroller_schema.Controllers (  
2     id uuid primary key,  
3     user_id uuid references user_schema.Users(id),  
4     plant_id uuid references social_media_schema.Plants(id),  
5     light float8,  
6     temperature float8,  
7     humidity float8,  
8     electrical_conductivity float8,  
9     soil_moisture float8,  
10    time timestampz,  
11    created_at timestampz default now()  
12 );
```

Abbildung 4.1: Controllers table in microcontroller_schema

From a practical perspective, maintaining this strict structural boundary noticeably improves overall system robustness. If the designated sensor reading service were to encounter an unexpected failure, the resulting downtime would likely not propagate sideways into the gamification or social interaction features.

4.2.2 Core Characteristics of Microservices in IoT

It must be acknowledged, however, that applying Microservices Architecture specifically to the Internet of Things reliably introduces a rather unique set of challenges. IoT systems tend to be inherently asynchronous and heavily event-driven. Moreover, they are forced to contend regularly with the wildly unpredictable nature of physical home environments.

Autonomy and Database per Service

One of the generally accepted fundamental principles of MSA is the concept of a “Database per Service”. This principle strongly dictates that microservices should be decoupled not just at the code boundary, but fundamentally at the data state level [?]. It attempts to prevent the hazardous scenario where modifying an underlying database table for one specific service inadvertently breaks the functionality of another depending on the same schema. In the context of “Plant Up!”, the Gamification Service therefore maintains its own entirely independent record of player statistics, keeping it carefully separated from the often chaotic, raw stream of incoming environmental sensor data.



Abbildung 4.2: Gamification player statistics

Scalability and Elasticity

IoT workloads, by their very nature, are characteristically intermittent. Long, quiet periods of sensor silence may be abruptly followed by sudden bursts of intense transmission activity whenever thousands of devices independently wake up to report their status changes. Utilizing MSA enables researchers to handle this variance somewhat gracefully by scaling the specific Ingestion Service horizontally (simply adding more ephemeral instances to absorb the load) without having to allocate similarly expensive, totally unnecessary resources to the more stable Social Service.

4.2.3 Internet of Things (IoT) Constraints and Hardware

At the very hardware foundation of “Plant Up!” sits the ESP32-S3 microcontroller, generally regarded as a highly capable System-on-Chip (SoC) produced by Espressif Systems [?]. Yet, despite offering quite decent processing power, it ultimately remains strictly bound by the harsh physical constraints of battery life. The choice of underlying software transmission protocol directly determines how long the hardware must remain in an energy-draining active state, thereby heavily dictating how long the battery actually lasts in an average household.

Power Consumption and Sleep Modes

The ESP32-S3 architecture provides several distinct power modes, each displaying vastly different energy consumption profiles. Understanding the nuances of these modes proved essential for the actual functional architectural design:

- **Active Mode:** Almost all integrated components, including the CPU, Wi-Fi radio antenna, and various peripherals, are fully powered up. In this state, the device predictably consumes somewhere between 160 mA and 260 mA [?]. Aggressively minimizing time spent in this mode is, quite frankly, critical for any hope of energy conservation.
- **Modem Sleep:** The main CPU remains active, allowing for script processing, but the energy-heavy radio is temporarily disabled. Power consumption drops noticeably to roughly 20–30 mA [?].

- Deep Sleep: The main CPU, the Wi-Fi module, and the primary RAM are all firmly powered down. Only the Ultra-Low Power (ULP) coprocessor and the basic Real-Time Clock remain weakly active. Current consumption drops precipitously to about 10 μA –150 μA [?].

The Wake-Up Tax

While deep sleep obviously provides very substantial energy savings, there is a catch: it introduces a noticeable “wake-up tax”. Upon waking up to perform a reading, the device finds itself forced to completely re-initialize its entire Wi-Fi module, hunt for the access point, and negotiate a fresh IP address on the local network. This clunky process alone typically demands between 1 to 3 seconds, consuming a taxing 150 mA on average, notably before a single byte of useful data is ever transmitted [?]. It follows that the specific choice of communication protocol (such as MQTT vs. REST) severely dictates the amount of additional handshake overhead this wake-up transition incurs.

4.2.4 Communication Protocols: MQTT vs. REST

Choosing the primary communication protocol forms one of the most critical structural architectural decisions for the data layer. It observably impacts energy efficiency, overall latency, and long-term reliability. This subsection attempts to briefly examine the two leading protocols under consideration for the “Plant Up!” sensor layer.

MQTT (Message Queuing Telemetry Transport)

MQTT fundamentally operates as a lightweight, binary messaging protocol, having been designed originally for scattered networks that suffer from unreliability and incredibly limited bandwidth [?]. Rather than direct point-to-point connections, it utilizes a distinctly different publish-subscribe routing model.

Architecture and Decoupling: Unlike the standard REST approach, which attempts to connect two endpoints directly and forcefully, MQTT comfortably introduces a central Broker to act as an intermediary. The small sensor (acting as the Publisher) merely flings data toward a designated topic (e.g., `plantup/sensor/01`) without retaining any knowledge whatsoever of which distant backend services are actually consuming that data. The dedicated Broker reliably handles the heavy lifting of message routing, pushing the data to all currently subscribed backend services (such as the main Ingestion Service). One might argue that this specific architecture neatly decouples the physical devices both in space (since they never need to know each other’s precise IP addresses) and critically in time (given that messages can be queued temporarily if a consuming service is currently offline or unreachable).

Working of MQTT in IoT

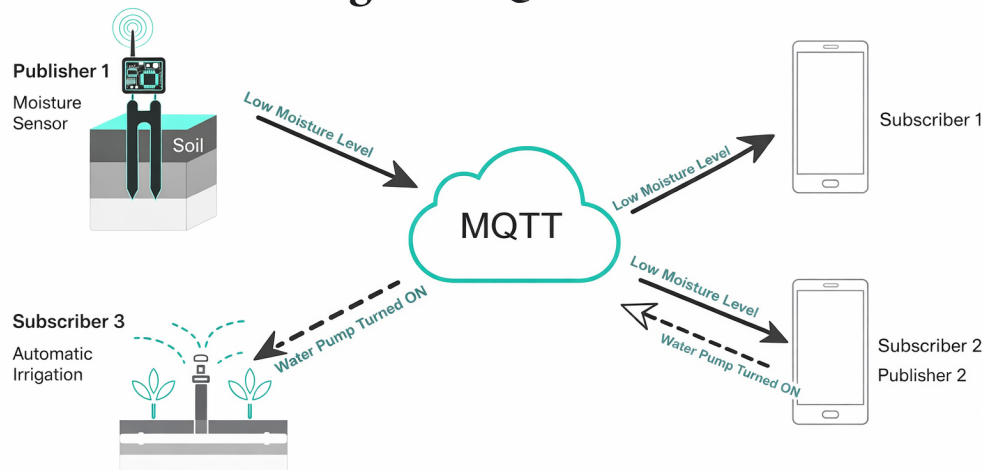


Abbildung 4.3: MQTT Publish/Subscribe Model: The Broker acts as a central hub, efficiently distributing messages from sensors to various services [?].

Packet Structure: Further reinforcing its efficiency, MQTT relies entirely upon a binary transmission format, which gracefully eliminates all bulky, unnecessary textual overhead. The fixed transmission header requires merely 2 bytes of space. This stands in stark contrast to traditional text-heavy HTTP headers, which can very easily accumulate to exceed several hundred bytes per single message.

Quality of Service (QoS): MQTT further differentiates itself by providing three sliding levels of distinct delivery assurance:

- **QoS 0 (At most once):** Effectively a blind fire-and-forget approach. It understandably consumes the least amount of energy, but assumes the risk that if a message is dropped mid-air, it is simply lost forever.
- **QoS 1 (At least once):** Prioritizes guaranteed delivery by enforcing a strict requirement for a return acknowledgment (PUBACK). This specific level generally proves most suitable for transmitting critical, actionable sensor data.
- **QoS 2 (Exactly once):** Relies on a rather cumbersome four-step handshake intended to guarantee exactly-once delivery. However, this level almost always introduces an untenable amount of wake-up overhead for severely constrained battery-powered devices.

REST (Representational State Transfer)

REST remains the ubiquitous standard architectural style of the modern web, comfortably utilizing highly standardized HTTP methods. It mostly follows a somewhat rigid, synchronous, and resource-oriented conceptual model.

Request-Response Model: REST strictly adheres to a traditional client-server behavioral paradigm. The roaming client explicitly issues a formatted request (like GET or POST) and then sits idle, forced to wait patiently for the distant server to respond before it is permitted to proceed with further tasks.

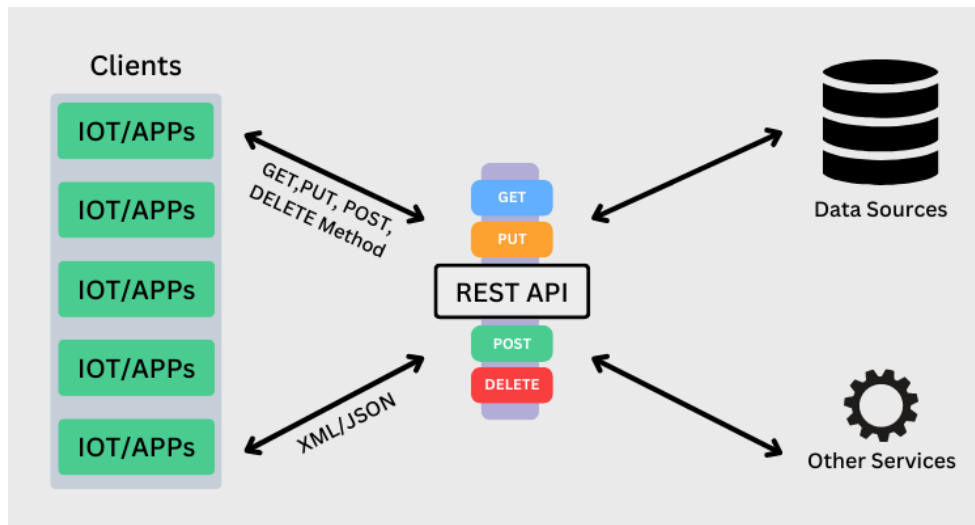


Abbildung 4.4: REST Request-Response Workflow: Stateless clients must establish a new connection for every request, incurring significant overhead [?].

Statelessness and Overhead: Under REST principles, the hosting server actively retains no memory state whatsoever between sequential requests. Consequently, every single transmission request must irritatingly carry all of its own necessary context (including heavy authentication tokens, diverse headers, and structural metadata) every single time. Furthermore, if the IoT device inevitably enters deep sleep between sequential readings, it is forced to endure a full, painful TCP three-way handshake, and frequently an additional TLS cryptographic handshake, for *every single isolated data transmission*. Looking at it closely, this mandatory connection overhead arguably renders REST significantly less energy-efficient than maintaining a partially persistent MQTT session for dealing with frequent, very small data packets.

4.2.5 Data Consistency and the CAP Theorem

In the realm of distributed systems, the widely referenced CAP theorem (specifically Consistency, Availability, Partition Tolerance) strictly establishes that all three functional properties simply cannot be guaranteed simultaneously in the harsh presence of a genuine network failure [?, ?].

For a project like “Plant Up!”, maintaining Partition Tolerance (P) is entirely non-negotiable from the outset, essentially due to the fact that scattered wireless home sensor networks are inherently fickle and broadly unreliable devices. This hard physical reality aggressively forces an architectural choice between pursuing either CP or AP behavior:

- **CP (Consistency + Partition Tolerance):** Under this model, incoming requests are flatly rejected if perfect data consistency cannot be absolutely guaranteed across

the network, which unfortunately introduces a high risk of outright data loss during a prolonged network disruption.

- AP (Availability + Partition Tolerance): Conversely, data is gracefully accepted and requests are cheerfully served to the user even if certain distant nodes are known to be temporarily out of sync with the master log.

“Plant Up!” ultimately embraces a robust AP design, leaning heavily into the concept of Eventual Consistency. From a practical perspective, it is deemed perfectly acceptable for a casual user to occasionally observe a static soil moisture value that is perhaps a few seconds out of date, provided the overarching mobile application remains entirely responsive and pleasantly stable. Under this framework, the deployed MQTT broker effectively serves as a resilient buffering system, safely holding incoming sensor messages during small network disruptions and ensuring they eventually reach the required downstream microservices layer intact.

With these somewhat dense theoretical foundations now firmly established, the heavily anticipated following section will finally present the concrete, implemented system architecture of “Plant Up!” in practice, clearly demonstrating how these abstract principles were applied to solve the real-world engineering problem at hand.

4.3 Plant Up! System Architecture and Implementation

4.3.1 System Overview and Vision

The “Plant Up!” platform fundamentally attempts to integrate distributed IoT hardware, a scalable cloud microservices backend, and a mobile application into one cohesive ecosystem. This setup effectively represents a practical realization of the Social Internet of Things (SIoT) concept outlined previously in the theoretical background. Ultimately, it enables plants to digitally communicate their environmental status to their caretakers. The primary objective behind this architecture is the collection of high-precision environmental data from custom edge devices, the subsequent synchronization of this data through the cloud, and its presentation to users via slightly gamified interactions.

To realize this broader vision, the architecture naturally had to address three rather critical requirements:

- Low-latency sensor synchronization: When a plant requires attention, it is generally expected that the user is notified promptly. The feedback loop between biological need and human action ought to be closed as rapidly as feasible.
- Energy-efficient operation: IoT devices intended for casual home use must realistically be designed for long-term deployment. This implies conserving battery power through intelligent deep sleep cycles and keeping radio usage to an absolute minimum.

- Distributed consistency: Because data is inherently distributed across User, Social, and Hardware domains, the system must maintain a relatively coherent state, even during the expected periods of localized network instability.

These requirements visibly informed almost every major architectural decision, extending from the specific sensors selected for the hardware up to the overarching structure of the microservices. As a result, the design follows a clearly separated multi-layered approach: the Edge Node, the Cloud Layer, and the Application Layer.

4.3.2 Hardware Layer: The Edge Node

The hardware layer effectively constitutes the physical sensing infrastructure of the project. It serves as the tangible bridge between the digital cloud backend and the physical environment of the plant. At the center of this design is the ESP32-S3 microcontroller, paired with a fairly comprehensive suite of sensors capable of measuring temperature, humidity, light, soil moisture, and electrical conductivity. Together, this configuration is intended to provide a relatively complete picture of the plant's current health. Due to the strict constraints of battery capacity, the device is programmed to spend the vast majority of its operational time in a deep sleep state. Consequently, its active cycle follows a rather strict sequence:

1. Acquisition: The microcontroller wakes up, temporarily powers the attached sensors, and captures a brief environmental snapshot.
2. Serialization: The recorded sensor readings are subsequently packed into a compact JSON format.
3. Transmission: The data packet is finally transmitted to the cloud (utilizing either MQTT or REST), immediately after which the device is instructed to return to its deep sleep mode.

Component Selection and Sensor Interface

Hardware selection typically involves a careful trade-off between technical capability and energy longevity. With these constraints in mind, the following components were chosen for the "Plant Up!" edge node.

Microcontroller: Espressif ESP32-S3 The ESP32-S3 acts as the central processing unit. It was selected primarily for its reasonable balance between computational power and energy efficiency. It features a dual-core processor alongside built-in Wi-Fi and Bluetooth connectivity. Of particular relevance here is its Ultra-Low Power (ULP) co-processor, which theoretically allows the main CPU to remain asleep while basic environmental monitoring continues in the background. Priced at approximately 20 Euro, it offers what can be considered a favorable cost-to-performance ratio for this context.

Soil Moisture Sensor: HiLetgo LM393 The HiLetgo LM393 (costing approximately 7.89 Euro) measures the dielectric permittivity of the soil. Unlike many lower-cost sensors that are notoriously prone to rapid corrosion, this resistive variant provides the reasonably reliable moisture readings necessary to accurately trigger watering notifications.

Light Sensor: HiLetgo BH1750 The BH1750 (approximately 11.39 Euro) is a standard digital I2C sensor providing relatively precise lux readings. Having this data enables the system to assess whether a plant is receiving adequate light exposure based on its species and to generate corresponding adjustments or notifications.

Electrical Conductivity (EC) Sensor: DFRobot Gravity V2 The DFRobot Gravity Analog EC Sensor (around 12.10 Euro) is utilized to measure the overall electrical conductivity of the soil, a metric heavily correlated with general nutrient concentration.

- *Relevance of EC:* The rationale here is that electrical conductivity correlates reasonably well with underlying nutrient levels. A plant might be perfectly well-watered yet still severely deficient in nitrogen. Integrating this sensor therefore attempts to elevate the system from functioning merely as a watering alarm into a broader, somewhat more comprehensive health monitor.

Power Supply and Sustainability To support remote, long-term operation, the main device pairs a standard lithium-ion battery with a small 0.5 W photovoltaic solar panel (costing approximately 3.48 Euro). This combined setup is designed to be largely self-sustaining under decent light conditions, while a standard USB-C port is still included as a practical fallback charging option.

Data Structure

Prior to transmission, all compiled sensor data is serialized into a structured JSON payload. This specific payload is designed to map directly to the `Controllers` table situated within the backend `microcontroller_schema`:

SENSOR PAYLOAD STRUCTURE

```

1 {
2   "user_id": "uuid",
3   "plant_id": "uuid",
4   "light": 350.5,
5   "temperature": 21.7,
6   "humidity": 45.3,
7   "electrical_conductivity": 1.2,
8   "soil_moisture": 23.8,
9   "time": "2025-01-12T10:15:30Z"
10 }
```

Abbildung 4.5: JSON payload sent by the edge node

The corresponding database table deployed in Supabase is shown below:

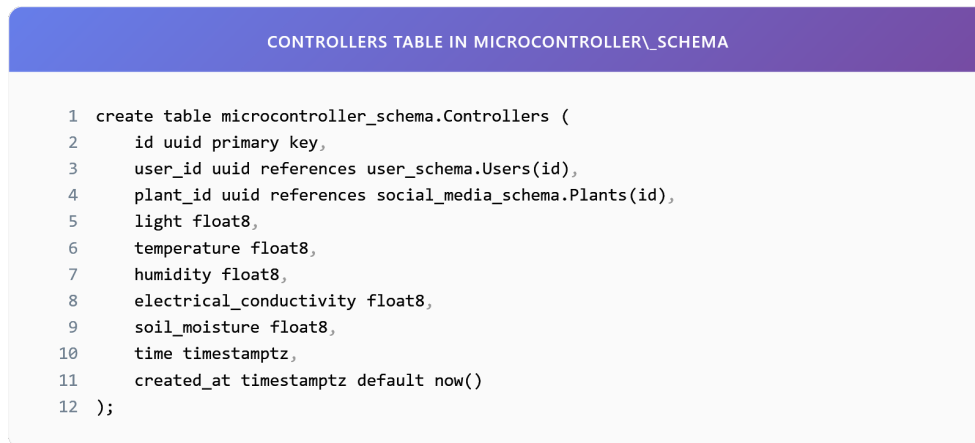


Abbildung 4.6: Controllers table receiving IoT data

A particularly critical design decision implemented at this stage involves timestamping every measurement at the source, that is, directly on the edge node itself. This approach ensures that even during extended periods of network unavailability, the backend can accurately reconstruct the historical data timeline based on the original recording time, rather than relying on a potentially delayed arrival time at the server.

4.3.3 Microservice Architecture Design

The backend infrastructure is fundamentally structured as a collection of microservices, organized largely around isolated Supabase schemas. In this architecture, each schema functions roughly as a Bounded Context [?], encapsulating a designated domain:

- **user_schema:** This area manages raw user profiles, activity streaks, and virtual wallets.
- **social_media_schema:** Responsible for community content, including user posts, comments, and public plant profiles.
- **microcontroller_schema:** Acts essentially as the centralized data warehouse for all incoming raw sensor readings.
- **gamification:** Houses the core game engine logic, including quests, overall progress tracking, and accumulated XP.

From a practical perspective, this strict separation provides notable development and operational benefits. For instance, any necessary modifications to the quest system can be carried out without risking inadvertent disruptions to the rather sensitive sensor ingestion pipeline. Consequently, failures isolated in one domain are highly unlikely to cascade into the others.

By way of example, the `Plants` table located in the social schema simply links a given user's plant to its ideal, scientifically established growing conditions, which are independently stored in `Plant_data`. This setup allows various other services to retrieve the

required botanical reference data without unnecessarily tangling dependencies within the social database.

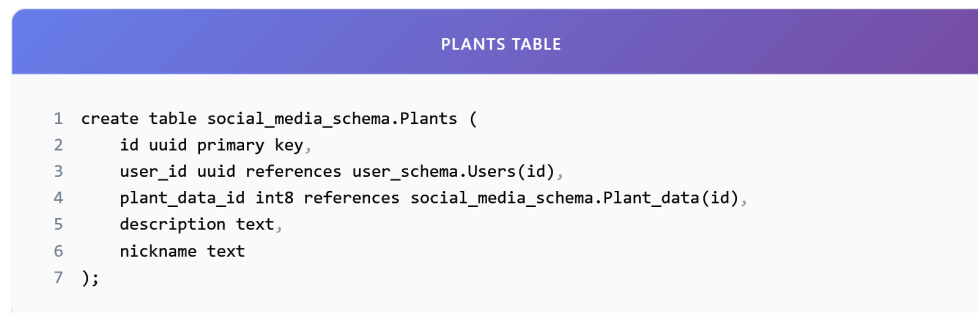


Abbildung 4.7: Plants table in social_media_schema

4.3.4 Real-Time Data Synchronization Mechanism

One of the central challenges encountered in the “Plant Up!” architecture is the ongoing attempt to balance real-time data delivery against the need for strict energy conservation on the remote sensor devices. While users generally expect to see fresh data reflected in the app quite promptly, the sensor nodes simply cannot afford the immense energy cost of maintaining entirely persistent network connections. It is therefore necessary to briefly explain the reasoning behind the adopted synchronization strategy.

During the planning phase, two primary contenders were evaluated for handling the core sensor-to-cloud communication:

- **REST (HTTPS):** Readily recognized as the standard approach of the modern web. It is highly structured and universally supported, which clearly makes it well-suited for traditional, user-facing API interactions. Nevertheless, when utilized for transmitting small, frequent sensor packets, the necessary connection overhead becomes substantial.
- **MQTT:** Originating as a protocol designed specifically with constrained IoT scenarios in mind. It is observably lightweight and operates on a publish/subscribe model, a structure that theoretically appears well-matched to the severe constraints of battery-powered microcontrollers such as the ESP32.

Ultimately, a hybrid approach was adopted in an attempt to leverage the distinct strengths of both protocols. The rationale framing this decision is largely as follows: MQTT is utilized exclusively for the uplink sensor-to-cloud communication path. This is because its lightweight binary protocol nature and persistent broker connection help minimize the energy expenditure required for each transmission cycle. Conversely, the user-facing mobile application continues to communicate with the backend via standard REST APIs, mostly because smartphones do not face the same extreme energy constraints and actively benefit from the request-response semantics typical of HTTP when loading complex user profiles or social feeds.

Following this architecture, the deployed IoT devices reliably publish their sensor data via MQTT to a central broker. A dedicated backend service then naturally subscribes to the relevant topics and securely persists the incoming data stream into the database. Only then does the mobile application periodically retrieve this processed data through standard RESTful endpoints.

A typical data ingestion flow under this model is illustrated below:

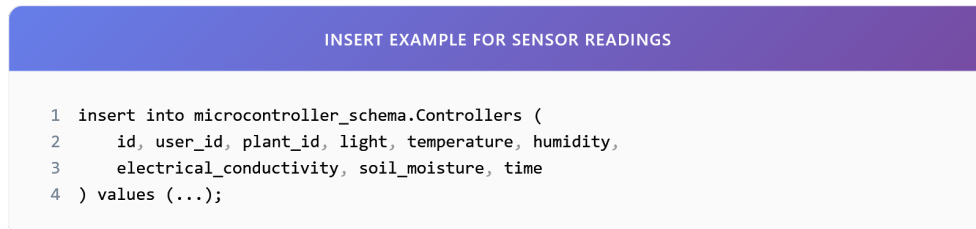


Abbildung 4.8: Insert operation for real-time sensor synchronization

4.3.5 User Engagement and Data Processing

It is worth noting that the social and gamification components integrated into “Plant Up!” are not intended merely as cosmetic additions; rather, they serve as the primary mechanism for actively driving user engagement over time. Naturally, this logic is housed within the `gamification` schema, where it continuously monitors the various data streams arriving from the other interconnected layers.

The system relies heavily on three core database tables:

- `Quests`: Essentially the master catalog of available challenges, incorporating tasks such as “Water your Monstera” or perhaps “Check the overall light level”.
- `User_requests`: Maintains a record of the individual user’s active progress on their explicitly assigned quests.
- `player_stats`: Stores the user’s cumulative behavioral statistics, including their overall XP and current level ranking.

For context, the `quests` table fundamentally defines the parameters of the available challenges:

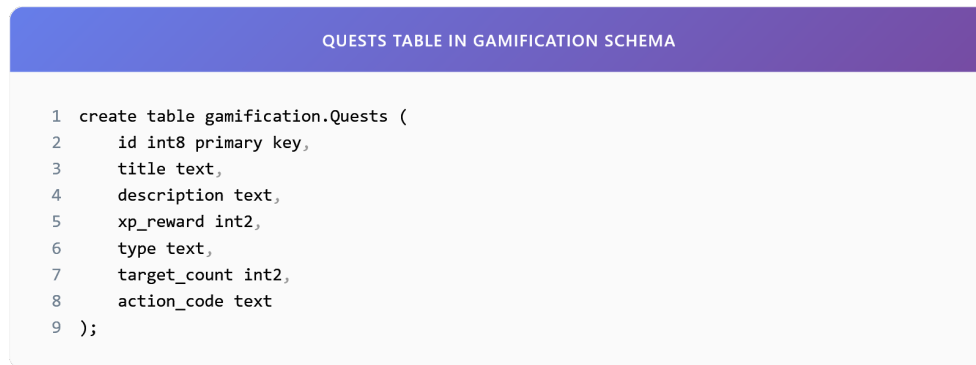


Abbildung 4.9: Quests table in gamification schema

The intended integration between the raw sensor layer and the higher-level gamification layer functions roughly as follows. When the sensor layer detects a noticeably sharp increase in soil moisture, the system logically infers that the user has recently watered the plant. The designated processing service then predictably triggers an immediate update to the user's internal quest progress. This creates a relatively direct link between physical, real-world action and digital, rewarding feedback.

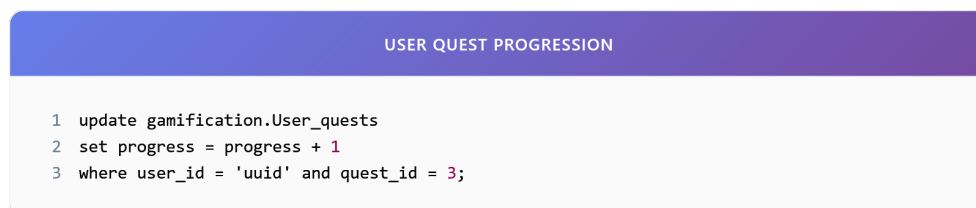


Abbildung 4.10: User quest progression

Once a specified quest goal is verifiably met (for instance, successfully watering a plant three times in a given week), the system proceeds to award the user with a predetermined amount of experience points.

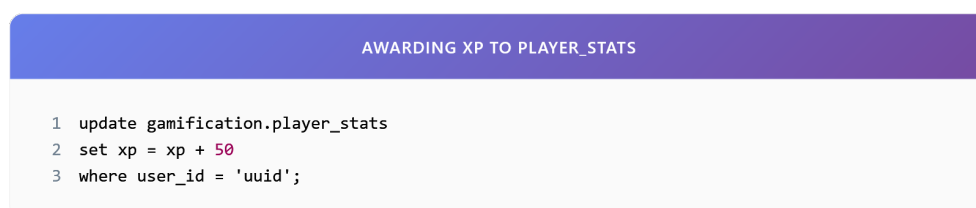


Abbildung 4.11: Awarding XP to player_stats

As one might argue, this distinctly modular design helps maintain a very clear operational separation between the higher-level engagement logic and the underlying hardware infrastructure. The various game mechanics, such as adjusting overall quest difficulty or running temporary seasonal events, can thus be modified entirely independently, without ever requiring a risky firmware update on the deployed edge devices.

While the architectural decisions presented throughout this section are certainly grounded in widely established theoretical principles, their real-world effectiveness still requires empirical validation. To provide this, the following section will detail a controlled

experimental evaluation designed to quantify the actual performance differences between MQTT and REST under realistic operating conditions, thereby directly addressing the research question posed earlier in the introduction.

4.4 Experimental Evaluation of MQTT vs. REST

4.4.1 Introduction to the Experiment

While the theoretical analysis presented in Section 2 suggested that MQTT offers distinct advantages over REST regarding protocol overhead and energy efficiency, theoretical insights alone are rarely sufficient to finalize architectural decisions for production environments. At this point, it is necessary to consider how these theoretical benefits translate into practice. Empirical, quantitative data is required to gauge the actual magnitude of the performance difference under realistic operating conditions. A closer examination of the system's behavior was therefore conducted through a targeted experiment. This evaluation attempts to directly address the research question posed initially, measuring the performance gap between MQTT and REST using the actual "Plant Up!" hardware and backend infrastructure.

4.4.2 Experimental Setup

To derive meaningful conclusions, the experimental environment needed to be reasonably standardized to ensure reproducibility. From a practical perspective, the setup was structured around three primary components:

1. **The Device (Edge Node):** The tests utilized a standard ESP32-S3 unit, essentially identical to the production hardware deployed in actual plant pots. For measurement purposes, the firmware was modified to isolate the protocol-specific transmission time from the rest of the boot process, employing high-precision microsecond timers.
2. **The Network:** Instead of relying on an idealized laboratory connection, the device was linked to a standard residential 2.4 GHz Wi-Fi router with an average signal strength (RSSI largely between -65 dBm and -75 dBm). This choice was mostly driven by the need to observe protocol behavior under the potentially flawed network conditions typically encountered by end users.
3. **The Backend:** All test data was routed to the live Supabase production database. This setup ensures that the latency measurements remain representative of the complete end-to-end path, encompassing network travel time, database insertion, and subsequent trigger execution.

In an effort to avoid confounding variables, such as fluctuations in sensor reading time,

a static, pre-defined JSON payload was transmitted during every trial. This approach guaranteed that the payload size stayed consistent across the 200 individual test runs.

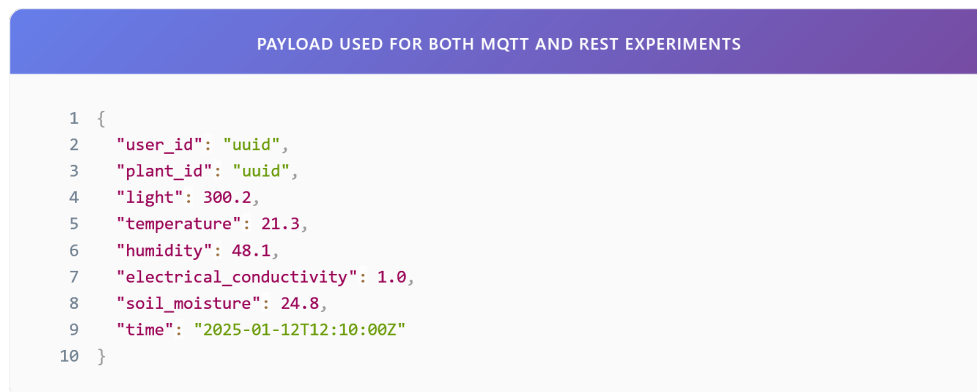


Abbildung 4.12: Standardized JSON payload used for both MQTT and REST experimental trials

4.4.3 Methodology and Metric Acquisition

To limit the influence of human handling variations, the data collection process was largely automated. The firmware was programmed to execute a repetitive measurement loop for each trial:

1. Wake Up: The microcontroller initiates its boot sequence from deep sleep.
2. Connect: A connection is negotiated with the local Wi-Fi router.
3. Send: The predefined JSON payload is serialized and transmitted, varying only in the underlying method (HTTP POST or MQTT PUBLISH).
4. Verify: The device then waits for a confirmation from the server, expecting either an HTTP 200 status or an MQTT PUBACK.
5. Sleep: Finally, the elapsed time is recorded locally, and the device is instructed to return to deep sleep without delay.

During this cycle, three primary metrics were monitored:

- End-to-End Latency: This captures the total time elapsed from the initial boot sequence to the receipt of server confirmation. It is a comprehensive metric, encompassing handshakes, serialization delays, and overall network propagation.
- Effective Payload Size: To understand the true protocol overhead, Wireshark packet analysis was employed. While a standard application payload might only be around 50 bytes, the total volume of transmitted data, once headers and protocol formatting are appended, often turns out to be noticeably larger.
- Reliability: This was quantified as the ratio of successfully persisted packets against total transmission attempts, evaluated over 100 distinct trials for each protocol.

Consistent packet reception and persistence were later verified by executing a targeted SQL query directly on the backend infrastructure.



Abbildung 4.13: SQL verification query used to validate data persistence

4.4.4 Experimental Results

An analysis of the collected data reveals a noticeable divergence in performance between the two implementations. Figure ?? summarizes the aggregated metrics gathered from the 100 test runs conducted per protocol.

GENERATED TABLE		
METRIC	MQTT	REST
Latency (ms)	185	612
Payload Size (bytes)	312	982
Success Rate (%)	98%	91%
Average Wake Duration (ms)	240	780

Abbildung 4.14: Comparative performance metrics of MQTT vs. REST under identical test conditions

The outcomes suggest that MQTT generally outperformed REST across the measured categories. The average end-to-end latency for MQTT was observed at 185 ms, whereas REST required 612 ms on average, amounting to a reduction of roughly 70%. A similar trend was visible regarding data usage: MQTT transmissions involved approximately 312 bytes per cycle, compared to around 982 bytes for REST.

4.4.5 Discussion

Viewed in context, these results tend to confirm the theoretical expectations regarding HTTP overhead in battery-constrained IoT applications. The recorded latency gap seems primarily driven by the connection establishment phase. For REST, each transmission mandates a full TCP three-way handshake followed by a TLS secure socket negotiation. Under these circumstances, the device ultimately spends more time and energy simply setting up the connection rather than transmitting the actual physiological data.

By contrast, MQTT, although still requiring initial negotiation, manages to omit much of the verbose text-based header information typical of HTTP, such as `User-Agent` and `Content-Type`. Its binary structure and arguably more streamlined handshake process contribute to a noticeably lower per-transmission overhead.

Beyond per-device efficiency, these differences in protocol handling introduce cascading scalability implications as the overall system footprint grows. In a microservices-based IoT architecture, managing concurrent inbound connections often evolves into a primary engineering constraint before sheer data volume does. At an initial deployment stage of roughly 100 active devices, both protocols essentially perform adequately from a backend perspective. The Supabase infrastructure easily absorbs the occasional bursts of REST-based HTTP requests without noticeable API degradation or unusual resource allocation.

As a hypothetical deployment scales toward 1,000 devices, however, the architectural friction inherent in HTTP becomes increasingly difficult to ignore. Because REST is fundamentally stateless and synchronous by design, each sensor node is compelled to establish a fresh TLS connection for every single measurement cycle. This introduces a persistent, cyclical connection overhead on the API Gateway, abruptly forcing the backend to continually allocate memory and processor cycles simply to negotiate, tear down, and rebuild network sockets. MQTT circumvents much of this friction through the use of persistent TCP connections. The central broker maintains an active, lingering state with the registered devices, effectively reducing the ingestion process to a continuous stream of lightweight binary packets rather than a chaotic flurry of independent, isolated requests.

At a localized peak of around 10,000 devices, these differences transition from being mere matters of optimization to dictating system survival. Attempting to ingest data from 10,000 sensors simultaneously via REST would inevitably risk triggering rate limits, exhausting API connection pools, and severely overwhelming the database insertion queues. Under an MQTT architecture, this massive influx of telemetry is gracefully decoupled. The broker naturally absorbs the transmission spikes, holding the incoming messages in a queue. The underlying backend microservices can then process this queue at their own sustained, predictable pace, safely persisting data to the `microcontroller_schema` without completely overwhelming the core database engine or risking locked tables.

Furthermore, this asynchronous design unlocks significant, highly tangible horizontal scaling potential. Should the ingestion service occasionally fall behind the broker during a massive influx of data—perhaps due to a sudden network reconnection event where thousands of devices transmit queued data simultaneously—additional, ephemeral worker instances can be spun up dynamically to drain the queue more rapidly. With REST, such a synchronized traffic spike impacts the database concurrently and directly, often leading to widespread transmission failures before auto-scaling rules can even react.

An additional, somewhat unintended consequence observed during the trials relates to reliability during periods of fluctuating network integrity. When the Wi-Fi environment became temporarily unstable, the tightly time-constrained HTTP requests were occasionally seen to time out and fail completely. MQTT's asynchronous architecture, paired

with its built-in Quality of Service (QoS) retry mechanisms, appeared noticeably better equipped to maneuver through partial connectivity, managing to successfully deliver packets through much narrower windows of network availability.

4.4.6 Energy Implications of Protocol Selection on ESP32-Based IoT Devices

While latency and system scalability certainly form central pillars of this architectural analysis, the most pressing practical constraint for a residential IoT environment arguably remains battery longevity. For the “Plant Up!” ecosystem, where the sensor node is designed to operate seamlessly and unobtrusively within a consumer’s home, the rate of energy depletion directly dictates the practical viability of the entire platform. At this stage, it is necessary to consider how the measured protocol overhead practically translates into physical power consumption. By analytically connecting transmission duration with the hardware’s specific power states, a clearer picture emerges of the long-term energy implications inherent in choosing between MQTT and REST.

To establish a baseline for this analysis, the power consumption profile of the ESP32-S3 microcontroller must be briefly examined. The device fundamentally operates across several distinct power states, each characterized by significantly different current draws. For the purpose of this estimation, realistic approximate values derived from the manufacturer’s documentation are utilized, though actual field performance will always exhibit minor variances based on temperature and battery degradation. During the *Active* mode, when the central processing unit is fully operational but the radio remains idle, the device typically consumes around 35 mA. However, the moment the Wi-Fi modem is powered on and actively negotiating or transmitting data, this current draw spikes considerably, often reaching an average of 240 mA [?]. Conversely, when the device enters its *Deep Sleep* state, with almost all peripherals aggressively powered down to conserve energy, the current consumption drops to roughly 10 μ A.

To quantify the energy expended during a single telemetry transmission cycle, the standard electrical energy formula can be applied:

$$E = I \times V \times t \quad (4.1)$$

In this equation, E represents the total energy consumed (measured in Joules), I signifies the electrical current drawn (in Amperes), V denotes the operating voltage (typically 3.3 V for the ESP32), and t specifically represents the duration of the active transmission state (in seconds). From this relationship, it becomes evident that because the operating voltage and the peak transmission current remain relatively constant during active Wi-Fi usage, the fundamental variable determining energy consumption is time. In other words, the total energy expended per cycle is almost entirely proportional to how quickly the device can successfully transmit its data and power down its Wi-Fi radio.

This observation brings the previously recorded experimental latency metrics into sharp, practical focus. During the physical evaluation, the average end-to-end latency for an

MQTT transmission was observed at 185 ms (0.185 s). Assuming an average active current of 240 mA during this phase, the energy consumed per MQTT update cycle can be reasonably estimated. Plugging these values into the formula yields:

$$E_{MQTT} \approx 0.240 \text{ A} \times 3.3 \text{ V} \times 0.185 \text{ s} \approx 0.146 \text{ Joules} \quad (4.2)$$

By contrast, the REST implementation recorded an average transmission latency of 612 ms (0.612 s). As previously noted, this delay is primarily due to the heavy overhead of establishing a fresh TCP connection and performing a full TLS handshake for every single isolated request. Applying the identical hardware parameters to this duration:

$$E_{REST} \approx 0.240 \text{ A} \times 3.3 \text{ V} \times 0.612 \text{ s} \approx 0.484 \text{ Joules} \quad (4.3)$$

The analytical results indicate a rather striking difference at the hardware level. A single REST transmission consumes nearly 3.3 times as much energy as a comparable MQTT transmission. This suggests that the verbose headers and synchronous nature of HTTP are not merely academic inefficiencies to be debated in software design; they generate a very literal, measurable drain on the device's physical battery chemistry. While the mathematical model assumes perfect transmission conditions, under practical deployment conditions where network congestion might cause retries, this energy gap would likely widen further. Every additional millisecond spent negotiating a secure socket translates directly into wasted Joules of finite energy.

The implications of this disparity compound significantly when extrapolated over a realistic operational timeline. For the purpose of long-term estimation, a typical use case for the "Plant Up!" sensor node can be assumed, where telemetry data is reported nominally once per hour. Over the course of a single year (comprising roughly 8,760 autonomous updates), the accumulated energy footprint expands substantially. Under an MQTT architecture, the transmission overhead alone would account for approximately 1,280 Joules per year. Under a strictly REST-based architecture, this identical workload would siphon away over 4,240 Joules annually—a difference that severely penalizes the device's operational lifespan.

This increased energy consumption naturally triggers a cascading series of secondary effects that ultimately impact long-term consumer usability. If a device utilizing REST drains its battery reserves noticeably faster than an optimally configured MQTT variant, the frequency of necessary battery replacements—or the required duration of solar recharging intervals—rises sharply. In a consumer context, frequent manual intervention entirely undermines the core premise of a "smart," autonomous sensor. A system that requires constant electrical maintenance shifts from being a convenient background utility to an active nuisance, drastically increasing the likelihood of user abandonment over time.

From an architectural standpoint, this analysis reinforces the notion that protocol efficiency holds vastly different levels of importance depending on exactly where it is applied within the system hierarchy. For the backend cloud servers, the difference between

parsing 312 bytes and parsing 982 bytes is structurally negligible, easily absorbed by the sheer processing capacity of modern, scalable cloud infrastructure. The server simply does not care about the extra fractions of a second it takes to parse an HTTP header. However, at the extreme edge of the network, within the stringent, physical confines of a battery-powered ESP32, these fractions of a second represent the difference between a product that lasts for several undisturbed months and one that frustrates the user by requiring a recharge every few weeks. It can therefore be reasonably concluded that optimizing protocol overhead at the device layer is perhaps the single most effective architectural intervention available for extending overall hardware longevity.

4.4.7 Conclusion

From the data gathered, it becomes evident that MQTT represents a more fitting protocol choice for the deeper sensor layer of the “Plant Up!” ecosystem. The reduction in both latency and data usage positions it favorably compared to the REST alternative. That said, REST maintains its relevance within the mobile application layer, where its familiar request-response mechanics are entirely adequate for fetching user profiles and related social content. Nevertheless, the ESP32-S3 hardware used at the edge simply cannot comfortably sustain the persistent overhead of HTTP for its high-frequency sensor reporting. Bearing these factors in mind, MQTT was ultimately selected as the standard protocol for telemetry ingestion, offering a more viable trajectory toward achieving acceptable battery longevity in a consumer-facing product.

Kapitel 5

Gamification. Gaming design patterns to keep people engaged in apps.

Author: Dennis Frenzel

5.1 Introduction

5.1.1 Background and Context

To get user hooked onto ones application is one of the most important factors for software success. Especially with the overflow of applications competing, keeping users motivated to perform regular tasks, particularly those that do not offer immediate rewards, is a challenge. Getting users interested in a normal activity is where gamification can help.

This thesis analyzes the concept of gamification and how it is applied within “Plant Up!”, a smart gardening system designed to assist users in caring for their plants.

5.1.2 Research Question

To systematically address these challenges this thesis poses the following primary research question:

How can gamification be effectively applied to a gardening system to keep users more engaged in a normal activity?

5.2 Theoretical Foundations of Gamification

5.2.1 Definition of Gamification

Gamification defines how game design elements and mechanics are integrated into non-game contexts while the main activity remains a non-game. The goal of introducing these elements is to increase user engagement and motivation by making otherwise boring or routine tasks more appealing [?, ?].

5.2.2 Difference between Gamification, Serious Games, and Games for Entertainment

It is important to note that gamification is not the same as serious games or games for entertainment. Although the terms sound similar, they describe different concepts. Gamification refers to the integration of game elements into non-game contexts in order to increase user engagement and motivation. Serious games are complete games that are explicitly designed for a primary purpose other than entertainment, such as education, training, or simulation. For example, a flight simulator used for pilot training or a surgery simulation used for medical training. Games for entertainment, on the other hand, are produced only for enjoyment, without any educational or functional objective [?].

5.2.3 Why Gamification Works in Non-Game Contexts

While many people question the effectiveness of gamification, research has shown that it can be an effective strategy for enhancing user engagement and motivation. This raises the questions: Why and how?

According to the Haufe Akademie, the effectiveness of gamification lies deep within the human psyche. It addresses the basic needs of autonomy, competence, and social relatedness, which form the foundation of human motivation. How gamification achieves this can be explained using three central mechanisms.

- **1:** Gamification integrates the human need for autonomy, competence, and social relatedness not merely through external rewards, but by actively supporting these needs.
- **2:** It redefines user interaction with a system. Instead of removing all difficult or boring tasks to maximize efficiency, gamification introduces voluntary challenges that encourage people to improve themselves. These tasks are transformed into engaging activities in which progress is clearly visualized and supported by immediate feedback mechanisms such as points and badges.
- **3:** Gamification uses social interaction to build a sense of community. Features such as cooperative challenges and leaderboards satisfy the desire to be noticed

by others, increasing attachment to the activity through competitive or collaborative motivation [?].

5.2.4 Motivation Theory

To understand why gamification can be effective in practice, it is necessary to examine the foundations of human motivation. Motivation is the psychological force that initiates, guides, and sustains goal-directed behavior. In the context of gamification, a key distinction is made between engaging in an activity because it is inherently rewarding (intrinsic motivation) and engaging in it to obtain an external outcome (extrinsic motivation) [?].

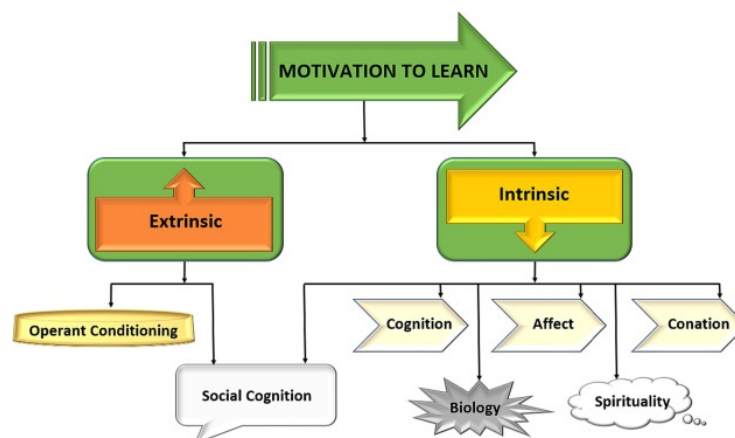


Abbildung 5.1: Classification of Motivation Types [?].

Intrinsic Motivation (The Engine of Engagement)

Intrinsic motivation refers to engaging in an activity because it is satisfying, enjoyable, or meaningful and is considered essential for long-term engagement [?]. According to Self-Determination Theory (SDT) [?], intrinsic motivation is supported when three basic psychological needs are fulfilled: Autonomy, Competence, and Relatedness.

When these needs are satisfied, individuals demonstrate increased curiosity, creativity, and persistence [?].

Self-Determination Theory (SDT) in Gamification

Self-Determination Theory establishes a framework for human motivation based on three fundamental psychological needs: autonomy, competence, and relatedness. Meeting these needs fosters self-determined motivation, which drives improved performance, persistence, and creativity. Failing to support them can significantly reduce motivation and well-being [?, ?].

Supporting Autonomy

Users should experience a sense of control over their actions. This is achievable with integrating optional tasks, multiple paths to success, and customization options [?].

Supporting Competence

Users require clear feedback that demonstrates progress and skill development, so that they can feel a sense of accomplishment and competence. It is mostly done by integrating experience points, progress indicators, adaptive difficulty, and immediate feedback loops [?].

Supporting Relatedness

Users are more engaged when they feel socially connected, hence integrating social feeds, cooperative goals, peer comparison, and mentorship features is a go to strategy [?].

Extrinsic Motivation (The Spark)

Extrinsic motivation involves performing an activity in order to receive a reward or avoid negative consequences. Common gamification elements such as points, badges, and leaderboards are examples of extrinsic motivators [?].

- **Controlling Extrinsic Motivation:** When rewards are perceived as chore-like, they can reduce intrinsic motivation, resulting in short-term compliance but reduced long-term engagement. A typical example is a streak-based system that pressures users to perform regularly, which can lead to stress and burnout.[?].
- **Autonomy-Supportive Extrinsic Motivation:** When rewards provide informational feedback or signal competence, they can be internalized and reinforce intrinsic motivation. Feedback such as positive reinforcement for progress or improvement is more effective than obligation-based incentives.[?].

Effective gamification strategies use extrinsic rewards not as the primary objective, but as a means of supporting intrinsic motivation [?, ?].

The Role of Extrinsic Rewards

Gamification is not merely about adding rewards, but about how they are implemented. Extrinsic motivators can be powerful drivers of engagement but may reduce creativity and encourage mechanical behavior. This can result in retention driven by obligation

rather than enjoyment [?]. When used carefully, extrinsic motivators can still provide short-term motivational boosts that foster intrinsic interests such as mastery or competition [?, ?].

Rewards that Support Intrinsic Motivation

Motivators that support intrinsic motivation are linked to a personal desire to master, explore, and enjoy an activity. They are the main motivator for long-term commitment, when implemented right, trying to keep them as informational feedback rather than controlling mechanisms is the key to succeeding [?] Often those are accomplished by integrating, badges, that indicate achievement and mastery, experience points, virtual currency for customization and titles [?].

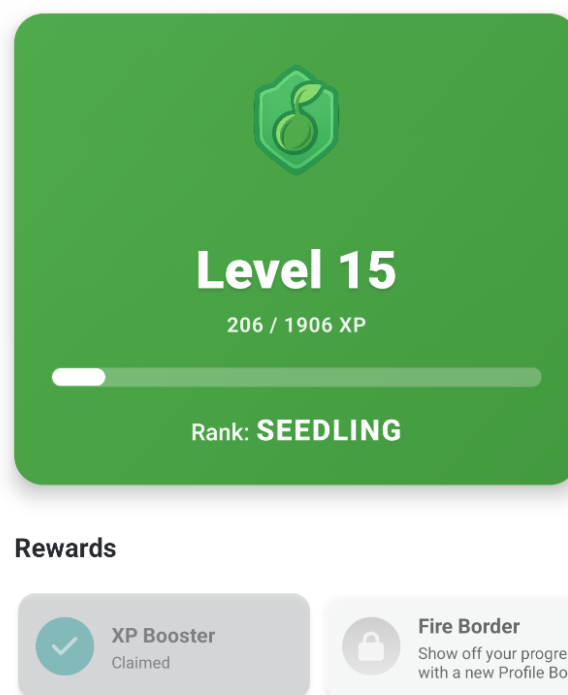


Abbildung 5.2: Level System and Rewards in Plant Up!.

Rewards that Undermine Intrinsic Motivation

Some rewards can be counterproductive when they seem controlling or worthless. [?] Such as over-rewarding insignificant actions, excessive penalties that induce stress, rankings without fairness or relevance, and when rewards replace genuine interest in the activity itself [?].

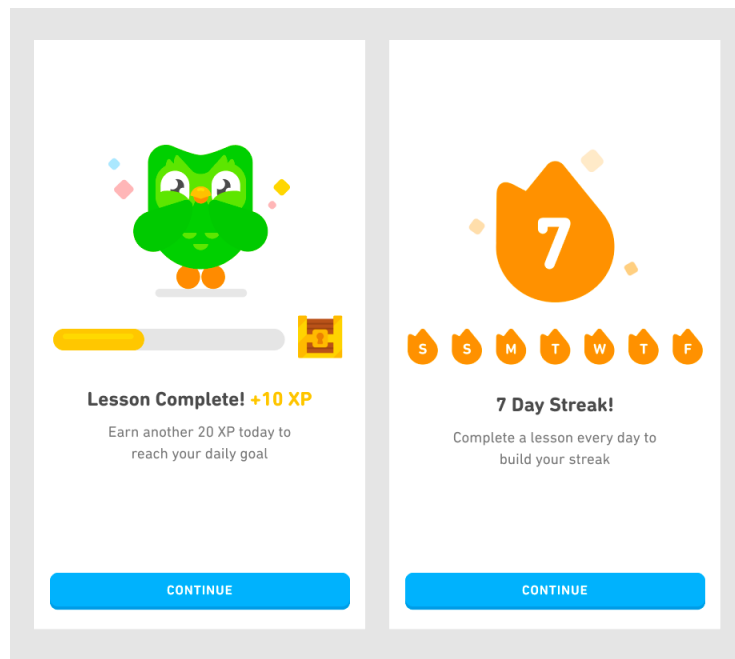


Abbildung 5.3: Duolingo Streak [?].

Attribution Theory

Attribution theory describes how individuals explain success and failure. In gamified systems, it is important that users get rewarded for their own effort and strategies rather than to luck or fixed ability. When implementing effort-based feedback it can increase motivation and encourage continued user engagement. Additionally, designing penalties and failure in such a way that it only seems temporary and improvable can lead to less frustration, hence making the user more likely to continue using the app [?].

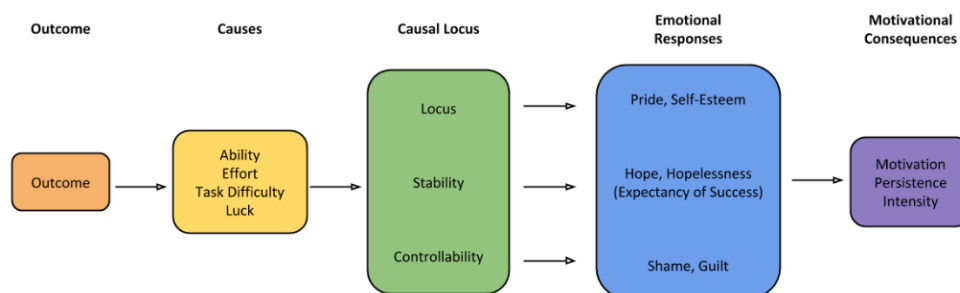


Abbildung 5.4: Attributional Process of Motivation.

Expectancy-Value Theory

Expectancy-value theory explains how what can motivate a user before engaging in an activity, those being: the expectation of success and the subjective value of the task. When a user thinks that they can succeed in an activity and values it, they are more likely to engage in it.

1. **Expectancy for Success:** Expectance for success can be defined with the question: "How likely is it that I will succeed in this activity?" In games and gamified systems, this belief is answered with a clear definition of goals, transparent rules, the difficulty of the task, and feedback provided during the activity. To make the user have a clear vision of their success, it is important to provide clear objectives, transparent mechanics, and visible indicators of progress. When the criterias of a "Is this challenge achievable?" question is met, the user is more likely to invest effort in it [?].
2. **Subjective Task Value:** Subjective Task Value, describes if the task is worth the effort, which is subjective for each user. This value consists of several components, including intrinsic value, the importance of performing well, usefulness for future goals, and effort, such as time investment. In games a value can be increased by making the user progress faster, reducing frustration or making those tasks relevant for future goals.[?].

5.2.5 Engagement and Habit Formation

Feedback Loops

A feedback loop is a important component of gamification, as it provides the users with immediate feedback on their progress and achievements.

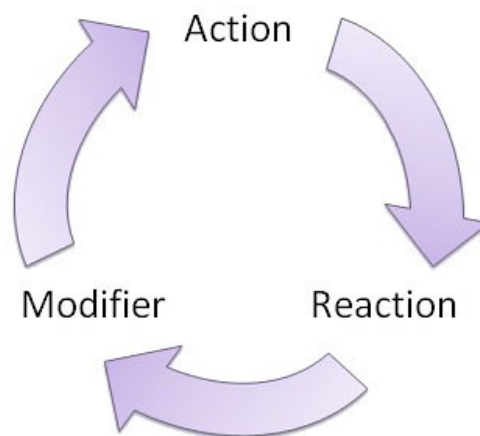


Abbildung 5.5: Feedback Loop [?].

It is important to take note of, that the feedback loop should consist of both positive and negative feedback.

Most people only like to receive positive feedback, such as: badges, experience points, virtual currency, titles, but it is important to take note of, that negative feedback can also be a motivator. Not only does a negative feedback, such as: penalties, warnings, or punishments, can be a wake up call for the user, to start improving their behavior, but it can also be a motivator to avoid those penalties. [?].

Short-term versus long-term engagement

Short-term engagement refers to the immediate satisfaction that users experience when they engage in an activity. Long-term engagement, on the other hand, refers to the sustained interest and motivation that users have for an activity over time. Short-term engagement is often driven by extrinsic motivators, such as rewards or recognition, while long-term engagement is driven by intrinsic motivators, such as the enjoyment of the activity itself.

Daily routines and behavioral reinforcement

Daily routines and behavioral reinforcement are important components of gamification, as they help users to develop and maintain healthy habits. Daily routines refer to the regular patterns of behavior that users engage in on a daily basis. Behavioral reinforcement refers to the process by which users are motivated to engage in a behavior because it has been reinforced in the past. Daily routines and behavioral reinforcement can be used to reinforce desired behaviors or to discourage undesired ones.

5.3 Gamification Design Patterns

5.3.1 Core Game Mechanics

- Points & XP
- Levels & progression curves
- Badges & achievements
- Virtual currency

5.3.2 Advanced Engagement Patterns

- Daily streaks
- Quests & challenges
- Time-based rewards
- Unlockables & rarity
- Loss aversion (streak freeze, decay)

5.3.3 Social Gamification

- Leaderboards (pros & cons)
- Social comparison
- Cooperative vs competitive design
- Community contribution & recognition

5.3.4 Failure States & Ethical Design

- Burnout risks
- Over-gamification
- Dark patterns (what to avoid)

5.4 Why Gamification Works for “Normal” Activities

This chapter answers the main thesis question directly by exploring how gamification transforms everyday activities.

5.4.1 Transformation of Mundane Tasks

- **Turning maintenance into progress:** Routine tasks often feel repetitive and unrewarding. Gamification provides a sense of progression for these necessary actions.
- **Making invisible progress visible:** Many real-world improvements happen too slowly to notice. Game elements quantify this progress immediately.

- **Micro-rewards for delayed real-world outcomes:** While the real-world benefit might take months (e.g., a plant blooming), virtual rewards provide instant gratification.

5.4.2 Feedback in Real-World Processes

- **Plants grow slowly vs. App feedback:** Biological processes are slow; the app bridges this gap with instant feedback loops.
- **Sensors for real-time gratification:** IoT sensors detect actions immediately, allowing the system to reward the user the moment they care for the plant.
- **Visual progress vs. real growth:** Virtual representations can grow and evolve visibly in the app even during periods where the real plant shows little visible change.

5.4.3 Emotional Attachment & Ownership

- **Avatars / Plant Identity:** Giving the plant a digital persona creates a relationship beyond simple object ownership.
- **Personal Investment:** Customization options increase the user's emotional stake in the system ('IKEA effect').
- **Long-term Bonding:** Reviewing history and milestones strengthens the connection over time.

5.5 Case Study: Gamification in Plant Up!

This section details the practical implementation of the theoretical concepts within the Plant Up! project.

5.5.1 Overview of Plant Up!

- **Smart gardening system:** An integrated solution creating a bridge between nature and technology.
- **IoT sensors:** Hardware sensors monitor soil moisture, light, and temperature in real-time.
- **Mobile application:** The user interface for monitoring data and interacting with the gamified elements.
- **Target users:** Hobby gardeners, tech enthusiasts, and people struggling to keep plants alive.

5.5.2 Gamification Concept in Plant Up!

The concept applies map theory to implementation:

- **XP for plant care:** Users earn Experience Points (XP) for performing correct care actions.
- **Streaks for daily check-ins:** Encouraging regular monitoring through consecutive daily login rewards.
- **Quests:** Specific challenges such as watering, fertilizing, or monitoring parameters.
- **Titles & Ranks:** Progression system from 'Seedling' to 'Evergreen' to 'Bloom'.
- **Coins & Lootboxes:** Virtual currency and random rewards to introduce variability.
- **NPC Guide ('Sprig'):** A character that provides tutorials, hints, and narrative context.

5.5.3 Mapping Design Patterns to Motivation

Each gamification element is designed to target specific motivational drivers:

- **Streaks → Habit Formation:** Builds daily routines through loss aversion.
- **XP → Competence:** Provides a quantifiable metric of skill and dedication.
- **Social Feed → Relatedness:** Allows users to share success and feel part of a community.
- **Titles → Identity & Status:** Public symbols of achievement and expertise.

5.6 Evaluation & Discussion

5.6.1 Expected Engagement Improvements

- **Increased daily active users:** Gamification features like streaks are expected to drive daily logins.
- **Longer session duration:** Interactive elements and progression systems encourage users to spend more time in the app.
- **Better plant care consistency:** Reminder quests and rewards aim to regularize care activities, leading to healthier plants.

5.6.2 Risks & Limitations

- **Users gaming the system:** Users might perform unnecessary actions just to earn points (e.g., over-watering).
- **Motivation drop after novelty:** The engagement spike from new features may fade (nnovelty effect").
- **Balance issues:** Ensuring rewards feel earned but attainable is difficult to fine-tune.

5.6.3 Comparison to Non-Gamified Apps

- **Traditional gardening apps:** Focus solely on information and scheduling.
- **Why they lose users:** Lack of immediate feedback and emotional hook leads to abandonment when the initial enthusiasm fades or the task becomes a chore.

Abbildungsverzeichnis

AI-Generated Figures

The following figures were generated using an AI image generation tool. The prompts used are listed below for transparency.

Figure ?? *Cross-section of a plant pot showing non-uniform soil moisture distribution. One side is visibly wetter (darker soil) near the surface where water was poured, while the opposite side remains dry (lighter soil). A soil moisture sensor probe is inserted on the dry side, illustrating how a single-point measurement can be misleading. Clean diagram style, white background, labeled arrows indicating wet zone, dry zone, and sensor position.*

Figure ?? *Professional flowchart for an academic thesis on white background with minimal style. Starting node: Sensor Data Received. Process box: Read Values (Soil Moisture, Temperature, Humidity, Light). Diamond decision: Compare against Thresholds. Three output paths: Green Zone (optimal, no action required), Yellow Zone (suboptimal, attention recommended), Red Zone (critical, immediate action required). All paths merge into: Generate Care Recommendation, then Display to User via App. Standard flowchart shapes, sans-serif font, colored zone boxes (green, yellow, red).*